পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৫ / ৩৭

কী চান — সেটা জিজ্ঞেস করবেন না

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.১ (রিকোয়্যারমেন্ট এলিসিটেশন)

ইন্টারভিউ · প্রশ্ন-সিঁড়ি · কাঁচা কথা থেকে ফিচার

৫ ভূমিকা · ১৫ সেশন · ৫৭ ফিচার

শুরু করুন প্রশ্ন দিয়ে

"আপনার কী কী ফিচার দরকার?" — এই প্রশ্নটাই ভুল

হাত তুলুন: আপনার প্রজেক্টে ব্যবহারকারীকে কী জিজ্ঞেস করবেন বলে ভেবেছেন?

মানুষ নিজের কাজ জানে।
সফটওয়্যারের সমাধান সে জানে না — সেটা আপনার কাজ।

"ফিচার কী চান" জিজ্ঞেস করলে উত্তর আসে **নকল সমাধান** — "আমার আগের সফটওয়্যারে যেটা ছিল", "একটা এক্সেল এক্সপোর্ট হলেই হয়"।
তাই প্রশ্নটা ঘোরাতে হয় কাজের দিকে: **"গত সোমবার কী করলেন, কোথায় আটকালেন?"**

শিক্ষকের নোট: কেউ "হ্যাঁ আগের সফটওয়্যারটাই চাই" বললে জিজ্ঞেস করুন — "আগেরটাতে সবচেয়ে বিরক্তিকর কী ছিল?" এই এক প্রশ্নেই আসল প্রয়োজন বেরিয়ে আসে, কারণ মানুষ অভিযোগ বলতে ভালোবাসে।

ধাপ ১

ইন্টারভিউয়ের তিনটি নিয়ম

১৫ মিনিটের ইন্টারভিউ — এই তিনটি না মানলে ফল শূন্য

১. গতকালের কথা বলান

"সাধারণত কী করেন" নয় — **"গত সোমবার সকালে ঠিক কী করলেন"**। নির্দিষ্ট দিন মনে করলে বানানো উত্তর কম আসে।

২. সমাধান নয়, কারণ

"কেন" তিনবার জিজ্ঞেস করুন। প্রথম "কেন"-এর উত্তর সাধারণত অজুহাত, তৃতীয়টায় আসল কারণ।

৩. যা নেই সেটাও জিজ্ঞেস করুন

"কোন তথ্য আপনি **দেখতে পান না**?" — পারমিশনের নকশা আসে এই প্রশ্ন থেকেই, ফিচার-তালিকা থেকে নয়।

ইন্টারভিউয়ের লক্ষ্য **তালিকা নয়** — লক্ষ্য হলো **একটা বাক্য**, যা দিয়ে আপনি তালিকাটা নিজে বানাতে পারবেন।

শিক্ষকের নোট: ২ নম্বর নিয়মটা ক্লাসে অভিনয় করে দেখান — একজন "শিক্ষক", আপনি তিনবার "কেন?" জিজ্ঞেস করছেন। তৃতীয় উত্তরেই সাধারণত "কারণ ভুল হলে দায় আমার ঘাড়ে পড়ে" — এটাই আসল প্রয়োজন, আর সেটা থেকে জন্মায় অডিট লগ।

ধাপ ২

প্রশ্ন-সিঁড়ি — পাঁচটি ধাপ, একই দিকে

প্রতিটি ধাপ আগেরটার উপর দাঁড়ায়; সিঁড়ি লাফ দিলে উত্তরও ফাঁকা আসে

১. কাজ
"গতকাল কী করলেন?"

২. বাধা
"কোথায় আটকালেন?"
"কত সময় লাগল?"

৩. কারণ
"কেন এটা সমস্যা?"
"কার ক্ষতি হয়?"
"কী হলে ঠিক হতো?"

৪. ব্যতিক্রম
"কখন নিয়ম ভাঙে?"
"ছুটির দিন?"
"দুই শিক্ষক একসাথে?"
"কেউ ভুল করলে?"

৫. শেষবার
"শেষ কখন ঘটেছে?"
"তখন কী করলেন?"
"কে টের পেল?"
"তা হলে করলেন কী?"

→ ফিচার নয়, প্রয়োজন বেরোয়

৪ নম্বর ধাপটাই সবচেয়ে বেশি বাদ পড়ে — অথচ ব্যতিক্রম থেকেই আসে "আজ নেওয়া হয়ে গেছে → এডিট মোড", "ভবিষ্যৎ তারিখ বন্ধ", "ছুটির দিনে সতর্কবার্তা"।

শিক্ষকের নোট: এই পাঁচটি ধাপ বোর্ডে লিখে দিন। ক্লাসের কাজে প্রতিটি জোড়া অন্তত সিঁড়ির ৪ ও ৫ নম্বর ধাপ পর্যন্ত যাবে — না গেলে ইন্টারভিউ অর্ধেক হয়েছে।

ধাপ ৩

পাঁচ ভূমিকা, চারটি আলাদা প্রশ্ন

একই প্রশ্ন সবাইকে করলে তিনজনের উত্তর নষ্ট হয়

ভূমিকা	প্রতিদিনের কাজ (জানতে হবে)	সবচেয়ে বড় ভয় (জানতে হবে)	যা দেখতে পায় না (জানতে হবে)	কতজনকে
শিক্ষক	হাজিরা · সিলেবাস · হোমওয়ার্ক · নম্বর	"ভুল হলে দায় আমার"	অন্য ক্লাসের নম্বর	৩–৫ জন
প্রতিষ্ঠান অ্যাডমিন	কাঠামো · শিক্ষক · ভর্তি · গ্রেডিং	"কেউ নিয়ম না মানলে?"	—	১–২ জন
কোঅর্ডিনেটর	নিজের বিভাগের দেখাশোনা	"কে হাজিরা নেয়নি জানি না"	বিল · অডিট লগ · সেটিংস	১–২ জন
শিক্ষার্থী	হোমওয়ার্ক দেখা · ফলাফল	"ফলাফল কখন আসবে"	অন্য কারও কিছু	২–৩ জন
প্রশাসক (প্ল্যাটফর্ম)	প্রতিষ্ঠান চালু · প্ল্যান	"ভুল প্রতিষ্ঠানে ডেটা"	ছাত্রের নম্বর	১ জন

মোট **৮–১৩টি ইন্টারভিউ**। এর কমে গেলে আপনি একজনের মতকে পুরো প্রতিষ্ঠানের মত ভাবছেন — এই ভুলটার নাম **নমুনা-পক্ষপাত**।

শিক্ষকের নোট: "যা দেখতে পায় না" কলামটাই এই টেবিলের সম্পদ। ছাত্ররা সাধারণত শুধু কাজ আর ভয় জিজ্ঞেস করে; তৃতীয় কলাম বাদ দিলে প্রতিটি পেজের পারমিশন পরে ঠিক করতে হবে — অর্থাৎ ক্লাস ৯-এ আবার সব করতে হবে।

ধাপ ৩ · স্ক্রিপ্ট

১৫ মিনিটের ইন্টারভিউ — সময়ের ভাগ

সময় না ভাগ করলে ১৫ মিনিটে শুধু ভূমিকা বলেই শেষ হয়ে যাবে

০–২ মিনিট
ভূমিকা
"গোপন রাখব"

২–৯ মিনিট · ৭ মিনিট
সিঁড়ির ১–৩ ধাপ: কাজ · বাধা · কারণ
গতকালের একটা দিন ধরে হাঁটুন

৯–১৩ মিনিট · ৪ মিনিট
ধাপ ৪–৫: ব্যতিক্রম · শেষবার
এখানেই আসল ফিচার বেরোয়

১৩–১৫ মিনিট
"যা দেখতে পান না?"
+ ধন্যবাদ · পরের প্রশ্ন

ভাগটা মন দিয়ে দেখুন: অর্ধেকের বেশি সময় যায় প্রথম তিন ধাপে, কিন্তু ফিচারের ৭০% আসে শেষ দুই ধাপ থেকে।
তাই ব্যতিক্রমের সময় কখনো কাটবেন না — ওখানেই ব্যবহারকারী নিজের সিস্টেম বানিয়ে ফেলেছেন।

শেষ ২ মিনিটে সবসময় একই প্রশ্ন: "আর কিছু আছে যা আমি জিজ্ঞেস করিনি?" — এখানে সবচেয়ে অপ্রত্যাশিত উত্তরটা আসে।

শিক্ষকের নোট: ক্লাসের কাজে এই ভাগটাই ব্যবহার করান — ১৫ মিনিটের টাইমার দিয়ে, প্রতিটি জোড়া একে অন্যকে ইন্টারভিউ করবে। সময় পার হলে থামিয়ে দিন; এতে সময়ের মূল্য বোঝা যাবে।

উদাহরণ ১

শিক্ষকের ইন্টারভিউ — পাঁচটি কাজ, একটা ক্রম

উত্তর থেকে সরাসরি তৈরি হয়েছে ম্যানুয়ালের অধ্যায় ৩

শিক্ষক বললেন (সারাংশ)

"সকালে ক্লাসে ঢুকে প্রথমে হাজিরা। তারপর সিলেবাস — কোন অধ্যায় পর্যন্ত গেছি মনে থাকে না। দিনে কী পড়ালাম লিখে রাখি খাতায়। হোমওয়ার্ক দিই, কে জমা দিল দেখি। মাস শেষে নম্বর তোলা — ওখানেই সপ্তাহ পার।"

ভয়: "নম্বর ভুল হলে অভিভাবক আমার সাথে কথা বলে।"

যা তৈরি হলো

উপস্থিতি	F-31 · F-32 · F-33 · F-34
সিলেবাস	F-36 · F-37 · F-38
হোমওয়ার্ক	F-39 · F-41 · F-42
পরীক্ষা	F-43 · F-44 · F-46 · F-47
রিপোর্ট	F-50 · F-51 · F-52

৫টি বাক্য → **২১টি ফিচার**। কারণ প্রতিটি দৈনন্দিন কাজের তিন-চারটি ধাপ থাকে।

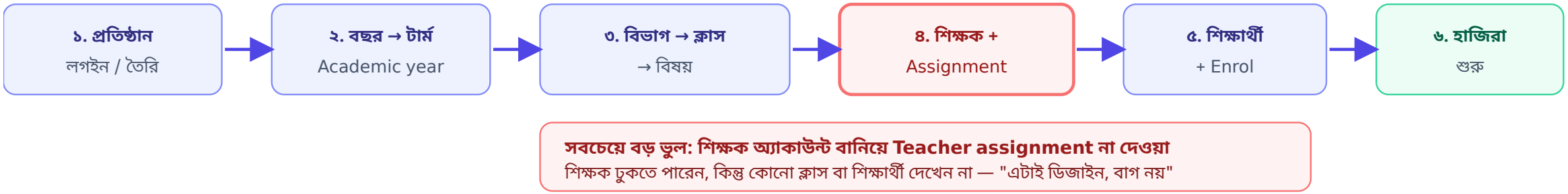
খেয়াল করুন: উত্তরে "সফটওয়্যার" শব্দটাই আসেনি। এসেছে **কাজ আর ভয়** — সেখান থেকেই ফিচার বেরোয়।

শিক্ষকের নোট: "ফলাফল প্রকাশের আগে অভিভাবক জানতে পারে না" — এই একটি ভয় থেকেই F-47 (publish) আর F-48 (শিক্ষার্থী শুধু publish-এর পরে দেখবে) এসেছে। ছাত্রদের বলুন: ভয় থেকে ফিচার বানানো easiest win।

উদাহরণ ২

অ্যাডমিনের ইন্টারভিউ — প্রথম দিনের ক্রম

"নতুন স্কুলে ঢুকে প্রথমে কী করবেন?" — উত্তরটাই সেটআপের নকশা



ইন্টারভিউয়ের সবচেয়ে দামি উত্তর প্রায়ই **"নতুনরা এখানে আটকায়"** — কারণ সেটা একসাথে দুটো জিনিস দেয়: সেটআপের ক্রম, আর একটা খালি-পর্দার ব্যাখ্যা।

শিক্ষকের নোট: এই ছয় ধাপ হুবহু ব্যবহারকারী ম্যানুয়ালের "প্রথম দিনের ক্রম" থেকে। ছাত্রদের বলুন নিজের প্রজেক্টে এমন একটা ছয়-ধাপের ক্রম লিখতে — না লিখলে ডেমোর দিনে নিজেই আটকে যাবে।

উদাহরণ ৩

কোঅর্ডিনেটরের ইন্টারভিউ — "আমি কী, না কে"

প্রশ্নের উত্তর থেকে জন্ম নিল একটা সম্পূর্ণ ভূমিকা

প্রশ্ন: "আপনি কে?"

প্রথম উত্তর: "আমি শিক্ষক, সাথে বিভাগের দেখাশোনা করি।"

দ্বিতীয় প্রশ্ন: "আপনার আর দশজন শিক্ষকের সাথে তফাত কী?"

আসল উত্তর: "আমি দেখি কে হাজিরা নিয়েছে, কে নেয়নি। কিন্তু বিল বা সেটিংসে আমার কিছু করার নেই।"

সিদ্ধান্ত: কোঅর্ডিনেটর পৃথক প্রতিষ্ঠান নয় — **একটা পদ**, যা অ্যাডমিন বসান।

"যা আপনার নেই" — সবচেয়ে দামি তালিকা

যা নেই	কার জন্য	কারণ
Audit log	অ্যাডমিন	পুরো প্রতিষ্ঠানের নিরাপত্তা-লগ
Workspace settings	অ্যাডমিন	কাঠামো বদলানোর অধিকার
Grading	অ্যাডমিন	স্কেল পুরো প্রতিষ্ঠানের
Coordinator scope	অ্যাডমিন	নিজের ক্ষমতা নিজে বাড়ানো যাবে না
Branding · বিল	অ্যাডমিন	পরিচয় ও টাকা প্রতিষ্ঠানের সিদ্ধান্ত

এই পাঁচটি লাইন থেকেই তৈরি হলো **পারমিশন ম্যাট্রিক্স** — শুধু "কে কী পারে" নয়, **"কে কী পারবে না, আর কেন"**।

শিক্ষকের নোট: সম্মানের সূক্ষ্মতা: "Coordinator scope" পাতা কোঅর্ডিনেটরের মেনুতেই নেই — শুধু লুকানো নয়, সার্ভারেও আটকায়। এই দ্বৈত সুরক্ষা ক্লাস ৯ ও পর্ব ৪-এ আবার আসবে।

ধাপ ৪

কাঁচা কথা → ফিচার আইডি: পাঁচটি আসল রূপান্তর

বাঁ দিকে ব্যবহারকারীর বাক্য, ডানে যা তৈরি হলো

ব্যবহারকারী যা বললেন	ফিচার	কীভাবে বদলালো
"সকালে পুরো ক্লাসের হাজিরা লিখতে ২০ মিনিট লাগে, মোবাইল থেকে লিখি"	**F-31**	বাল্ক বোতাম (সবাই উপস্থিত → যাদের অনুপস্থিত শুধু বদলান) + মোবাইল লেআউট
"একই দিনে দুইবার সেভ চাপলে ডুপ্লিকেট হয়ে যায়"	**F-31**	`unique(student, class, subject, date)` + দ্বিতীয়বার হলে **এডিট**
"কে হাজিরা নেয়নি — মাসের শেষে জানতে চাই"	**F-35**	উপস্থিতি **পর্যবেক্ষণ** পর্দা — কোঅর্ডিনেটরের নিজের, শিক্ষকের নয়
"ফলাফল কখন দেবে কিছু জানি না, বারবার দেখতে আসি"	**F-47 · F-48**	**প্রকাশ** একটা আলাদা কাজ; শিক্ষার্থী কেবল প্রকাশের পরেই দেখবে
"ভুল হলে কে বদলাল, জানতে চাই"	**F-54**	সংশোধনে **কারণ** বাধ্যতামূলক → অডিট লগে থেকে যায়

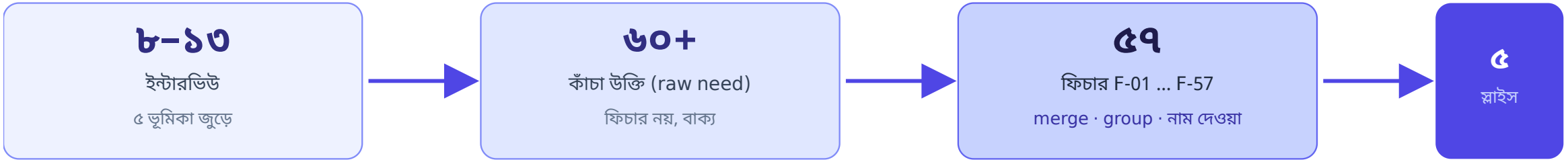
নিয়ম: প্রতিটি ফিচারের পাশে **যে বাক্যটা থেকে এসেছে** সেটা লিখে রাখুন। পরে "এই ফিচারটা কে চেয়েছিল?" প্রশ্নের উত্তর থাকবে।

শিক্ষকের নোট: দ্বিতীয় সারিটা সবচেয়ে শিক্ষণীয় — ব্যবহারকারী "বাগ" বলেছেন, আমরা "ডিজাইন" দিয়েছি (unique + এডিট)। ছাত্রদের জিজ্ঞেস করুন: এখানে বাগটা আসলে কোথায় ছিল? উত্তর: ডেটাবেসে নিয়ম ছিল না, শুধু ফর্মে ছিল।

ধাপ ৫

১৫ সেশন → ৫৭ ফিচার

ইন্টারভিউয়ের সংখ্যা আর ফিচারের সংখ্যা সমান হয় না — কারণ ব্যবহারকারী ভাগে ভাগে ভাবেন



মাঝের ধাপে তিনটি কাজ: একই কথা একত্র করা · ধাপগুলো দলে ভাগ করা · প্রতিটিকে নাম ও আইডি দেওয়া

৬০+ কাঁচা উক্তি থেকে ৫৭ ফিচার — অর্থাৎ প্রায় প্রতিটি উক্তিই একটা ফিচার, কিন্তু তার আগে ২০টি ডুপ্লিকেট বাদ দিতে হয়েছে

> ডুপ্লিকেট বাদ দেওয়াই সবচেয়ে বড় কাজ — পাঁচজন শিক্ষক পাঁচভাবে একই কথা বলেন; সেটা **একটা** ফিচার, পাঁচটা নয়।

শিক্ষকের নোট: বোর্ডে দুটি সংখ্যা লিখুন — কাঁচা উক্তি আর ফিচার। ছাত্ররা দেখবে দুটো প্রায় সমান, কিন্তু ফিচারের তালিকায় প্রতিটি লাইন একবারই আছে।

সতর্কতা

যে পাঁচটি প্রশ্ন করবেন না

বাঁ দিকের প্রশ্নে উত্তর আসে "হ্যাঁ", ডান দিকের প্রশ্নে আসে গল্প

× ভুল প্রশ্ন	✓ ঠিক প্রশ্ন	কারণ
"আপনার কি ড্যাশবোর্ড দরকার?"	"সকালে ঢুকে প্রথমে কী দেখতে চান?"	প্রথমটা সমাধান চাপিয়ে দেয়
"আপনি কি মোবাইলে ব্যবহার করবেন?"	"শেষবার কখন মোবাইল থেকে কাজ করেছেন? কী করছিলেন?"	দ্বিতীয়টায় আসল প্রেক্ষাপট
"আপনি কি অডিট লগ চান?"	"শেষ কখন কেউ নম্বর বদলেছে টের পেয়েছেন?"	প্রয়োজন থাকলে নিজেই বলবেন
"এটা কি ভালো হবে না?"	"এটা হলে আপনার কাজ কমবে না বাড়বে?"	সম্মতির চাপ এড়ানো
"আপনার সমস্যা কী?"	"গতকাল কোন কাজটা সবচেয়ে বেশি সময় নিল?"	"সমস্যা" জিজ্ঞেস করলে সাধারণ অভিযোগ আসে

ভদ্রতা একটা ফাঁদ: মানুষ **না বলতে চায় না**। তাই "দরকার?" প্রশ্নের উত্তর কখনো "না" আসে না — আর সেই "হ্যাঁ"-এর পেছনে কোনো কাজ থাকে না।

শিক্ষকের নোট: ক্লাসে এক জোড়া দিয়ে দুটো প্রশ্নই করিয়ে দেখান — পার্থক্যটা নিজের কানে শুনলে আর ভুলবে না।

ধাপ ৬

নোট রাখার ফরম্যাট — পাঁচ কলাম

ইন্টারভিউ শেষে ১০ মিনিটেই লিখে ফেলুন, নইলে স্মৃতি বানিয়ে বলবে

তারিখ	কে	উক্তি (হুবহু)	দেখা সমস্যা	সম্ভাব্য ফিচার
২৮/১০	শিক্ষক · রফিক	"২০ মিনিট লাগে, মোবাইলে ভুল চাপ পড়ে"	হাজিরা ধীর · মোবাইলে ছোট বোতাম	F-31 বাল্ক + মোবাইল
২৮/১০	শিক্ষক · রফিক	"দুইবার সেভ করলে দুটো রেকর্ড"	ডেটাবেসে নিয়ম নেই	F-31 unique + এডিট
২৮/১০	কোঅর্ডিনেটর · সুমি	"কে নেয়নি জানতে মাস পার হয়"	পর্যবেক্ষণের পর্দা নেই	F-35 oversight
২৯/১০	শিক্ষার্থী · নীলা	"ফল কখন আসবে জানি না"	প্রকাশের অবস্থা দৃশ্যমান নয়	F-47 → F-48
২৯/১০	অ্যাডমিন · হাসান	"কে বদলাল সেটা জানা দরকার"	কোনো রেকর্ড থাকে না	F-54 audit log

হুবহু কেন: আপনার ভাষায় লিখলে পরে নিজেই নিজের অনুমান পড়বেন, ব্যবহারকারীর কথা নয়।

"সম্ভাব্য" শব্দটা জরুরি: সব উক্তি ফিচার হয় না — ইন্টারভিউয়ে সব নয়, পরে ছাঁটবেন।

শিক্ষকের নোট: এই পাঁচ কলামের খাতা ক্লাস ৬-এ টানা হবে — সেখানেই "সম্ভাব্য ফিচার" কলাম থেকে ৫৭ ফাংশনের ক্যাটালগ দাঁড়াবে।

দক্ষতা

উত্তর শুনে যা করতে হয় — তিনটি ছাঁকনি

ইন্টারভিউ শেষ, এখন কাঁচা কথা ছেঁকে ফিচার বানানো

🔁

১. একত্র করা

পাঁচজন শিক্ষকের "হাজিরা ধীর" — **একটাই** ফিচার। প্রতিষ্ঠানভেদে ভিন্ন হলে **সেটিং**, ফিচার নয়।

✂️

২. আলাদা করা

"ফলাফল দেখতে চাই" আসলে তিনটি কাজ: **হিসাব** · **প্রকাশ** · **দেখা**। তাই তিনটি ফিচার, একটা নয়।

🚫

৩. বাদ দেওয়া

"SMS পাঠান" — খরচ আছে, কিন্তু MVP-তে দরকার নেই। **"কখনো নয়"** বাক্সে যাবে, কারণ লিখে রাখতে হবে।

পরীক্ষা: প্রতিটি ফিচারকে **একজন ব্যবহারকারীর এক বাক্যে** বলা যায় কি? না গেলে ফিচারটা তখনো দুটো ফিচার — ভাঙুন।

শিক্ষকের নোট: "আলাদা করা" নিয়মটা ক্লাস ৬-এর ভিত্তি — একটা উক্তি থেকে তিনটি ফিচার বেরোলে তিনটির আইডি আলাদা হবে, আর স্লাইসে তিনটি আলাদা সময়ে তৈরি হবে।

বাস্তব

ইন্টারভিউ যেখানে সত্যিই বাধে — চারটি ঘটনা

প্রজেক্টের ফাইল থেকে, অনুমান নয়

পরিস্থিতি	যা ধরা পড়ল	যা করা হলো
শিক্ষক অ্যাকাউন্ট খুলে ঢুকলেন, কিছু নেই	Assignment ছাড়া ভূমিকা অর্থহীন	সেটআপের ক্রম ম্যানুয়ালে ৬ ধাপে লেখা · খালি পর্দায় ব্যাখ্যা
কোঅর্ডিনেটর নিজের ক্ষমতা বাড়াতে চাইলেন	পদটি প্রতিষ্ঠানভিত্তিক, ব্যক্তিভিত্তিক নয়	"Coordinator scope" পাতা শুধু অ্যাডমিনের, সার্ভারেও আটকায়
শিক্ষক দুই প্রতিষ্ঠানে পড়ান	বিল কার নামে?	সাবস্ক্রিপশন প্রতিষ্ঠানে (`8a25724`), শিক্ষক শুধু সদস্য
এক প্রতিষ্ঠান নিজের ডোমেইন চাইল	ভুল ডোমেইনে অন্য প্রতিষ্ঠানের লোগো যেতে পারে	নিজে সেট করার অধিকার নেই — প্ল্যাটফর্ম টিম করে

চারটিরই শুরু **একটা প্রশ্নে**, একটা ফিচার-অনুরোধে নয়। তাই ইন্টারভিউয়ের ফল ফিচার-তালিকা নয় — **নিয়মের তালিকা**ও।

শিক্ষকের নোট: শেষ সারিটা সবচেয়ে অপ্রত্যাশিত — "কাস্টম ডোমেইন" শুনতে ফিচার মনে হয়, আসলে নিরাপত্তার সিদ্ধান্ত। ছাত্রদের বলুন: ব্যবহারকারীর অনুরোধ শুনে প্রথম প্রশ্ন "এটা কার ক্ষতি করতে পারে?"

আলোচনা

চারটি প্রশ্ন — ইন্টারভিউয়ের আগে

উত্তর দিয়ে আজকের ক্লাস শেষ করুন

১. আপনার প্রজেক্টে সবচেয়ে কম **৩ জন** কে ইন্টারভিউ করবেন? নাম লিখুন — নাম না লিখলে কখনো হবে না।

২. সিঁড়ির ৪ ও ৫ নম্বর ধাপের (ব্যতিক্রম, শেষবার) জন্য আপনি ঠিক কী প্রশ্ন করবেন?

৩. আপনার প্রজেক্টে "যা দেখতে পায় না" — এই কলামে কার জন্য কী লেখা হবে?

৪. আপনি ইতিমধ্যে যে ফিচারটা নিজের ইচ্ছায় ঠিক করেছেন — সেটার পেছনে **কোন ব্যবহারকারীর কোন বাক্য** আছে? না থাকলে সেটা ফিচার নয়।

শিক্ষকের নোট: ৪ নম্বরের উত্তর বেশিরভাগের কাছে খালি থাকবে — এটাই স্বাভাবিক, কারণ এখনো কেউ ইন্টারভিউ করেনি। এটাই ক্লাস ৬-এর প্রবেশপথ: ফিচার তালিকা ইন্টারভিউ থেকে আসে, মাথা থেকে নয়।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
"কী চান" জিজ্ঞেস করলে নকল সমাধান আসে — কাজ ও ভয় জিজ্ঞেস করতে হয়	শিক্ষকের ৫টি দৈনন্দিন কাজ → ২১টি ফিচার, উত্তরে "সফটওয়্যার" শব্দই আসেনি
পাঁচ ধাপের প্রশ্ন-সিঁড়ি — ৪ ও ৫ বাদ দেওয়া যায় না	"দুইবার সেভ" → unique + এডিট মোড (F-31)
"যা দেখতে পায় না" — পারমিশনের আসল উৎস	কোঅর্ডিনেটরের ৫টি "যা নেই" → পারমিশন ম্যাট্রিক্স
উত্তর থেকে আসে নিয়মও, শুধু ফিচার নয়	কাস্টম ডোমেইন = নিরাপত্তার সিদ্ধান্ত, ফিচার নয়
প্রতিটি ফিচারের পাশে তার উক্তি রাখুন	ক্যাটালগে `F-31`-এর পাশে "purpose" লাইন — প্রশ্নের উত্তর সেখানেই

শিক্ষকের নোট: তৃতীয় লাইনটা পর্ব ৩–৪-এ বারবার ফিরে আসবে; ছাত্রদের এখনই মনে করিয়ে দিন যে পারমিশন ডিজাইন করার সময় নতুন কিছু ভাবতে হবে না — শুধু এই কলামটা দেখতে হবে।

০৬

পরের ক্লাস — "৫৭ ফাংশন, এক তালিকা"

কাঁচা উক্তি থেকে ফাংশন আইডি · অ্যাপ × ভূমিকা ম্যাপ · স্কোপ ফ্রিজে সই

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৬ / ৩৭

৫৭ ফাংশন, এক তালিকা

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.২ (ফাংশনাল রিকোয়্যারমেন্ট)
ফাংশন আইডি · অ্যাপ × ভূমিকা ম্যাপ · স্কোপ ফ্রিজে সই

১১টি অ্যাপ · ৫৭টি আইডি · ৬টি ভূমিকা

শুরু করুন প্রশ্ন দিয়ে

আপনার প্রজেক্টে কয়টা ফিচার? — সংখ্যাটা জানেন?

হাত তুলুন: কে বলতে পারবেন — ঠিক কতটা?

"অনেকগুলো আছে" — এটা সংখ্যা নয়।

আর সংখ্যা না জানলে স্কোপও নেই।

আজকের ক্লাসের ফলাফল একটাই জিনিস: **একটা তালিকা** — যার প্রতিটি লাইনের একটা **আইডি**, একটা **উদ্দেশ্য**, একটা **রুট**, আর একটা **মালিক-ভূমিকা**।
তালিকা ছাড়া "প্রজেক্ট ৭০% শেষ" বলার কোনো মানে নেই।

শিক্ষকের নোট: কেউ "আমার ২০টা ফিচার" বললে জিজ্ঞেস করুন — "নাম বলুন।" ১৪-তে থেমে যাবে। এই মুহূর্তটাই ক্লাসের সূচনা: মুখে বলা সংখ্যা সবসময় বেশি, তালিকায় কম।

ধাপ ১

একটি ফিচারের চারটি অংশ — আর আইডির নিয়ম

প্রজেক্টের ক্যাটালগে প্রতিটি ফিচার হুবহু এই গঠনে লেখা

```
F-33  Attendance register (by period)
role  : TEACHER
route : /w/<id>/attendance/register/
why   : একই দিনের সব পিরিয়ড এক পাতায় দেখা,
        কে নিয়েছে আর কে নেয়নি বোঝা
do    : ক্লাস · তারিখ · বিষয় বাছুন
        পিরিয়ড অনুযায়ী তালিকা দেখুন
data  : AttendanceRecord(student, class,
        subject, date) — একটি সারি = এক দিন
state : কোনো রেকর্ড নেই → খালি অবস্থার বার্তা
        লকড টার্ম → সম্পাদনা বন্ধ
```

আইডির চারটি নিয়ম

১. নম্বর **কখনো পুনর্ব্যবহার নয়** — ফিচার বাদ গেলেও আইডি মরে যায়।

২. **কখনো পুনঃক্রম নয়** — F-33 সবসময় রেজিস্টারই থাকবে।

৩. না-তৈরি ফিচারও **আইডি পায়** — নইলে পরিকল্পনা আর প্রমাণ মেলানো যায় না।

৪. আইডি **লেখায় ব্যবহার হয়** — কমিট, টেস্ট, ডকুমেন্ট, স্ক্রিনশটের নামে।

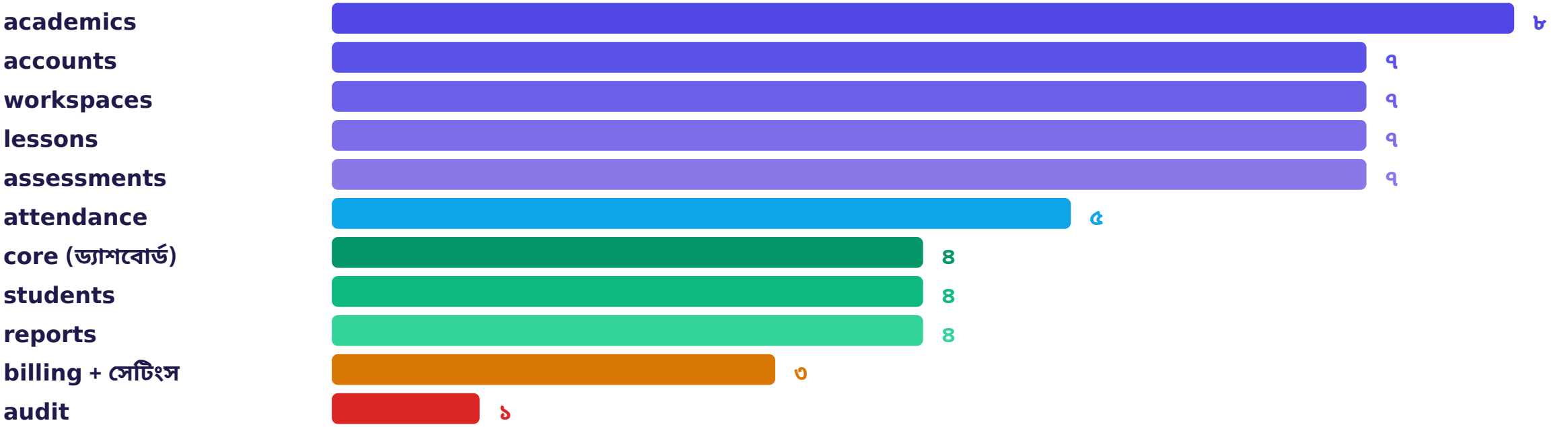
আইডি বদলালে পুরনো কমিট, পুরনো টেস্ট, পুরনো ডকুমেন্ট — সবের সংযোগ ছিঁড়ে যায়।

শিক্ষকের নোট: ৩ নম্বর নিয়মটা প্রথমে অদ্ভুত লাগে — "যেটা বানাব না, তার আবার আইডি কেন?" উত্তর: কারণ স্কোপের প্রশ্নটা "কোনটা বানাব" নয়, "কোনটা বানাব না আর কেন" — বাদ দেওয়া ফিচারও হিসাবের অংশ।

ধাপ ২

১১টি অ্যাপে ৫৭টি ফিচার — কোথায় কত

ব্লক লম্বা মানে ওই অ্যাপে কাজ বেশি — অর্থাৎ ওখানেই বাগও বেশি হবে



academics-এ ৮টি — অথচ এটা সিস্টেমের ভিত্তি। ভিত্তিতে ভুল হলে উপরের সবকিছু ভাঙে, তাই এই অ্যাপের টেস্ট সবচেয়ে বেশি চাই।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "কোন অ্যাপটা সবচেয়ে আগে বানাতে হবে?" উত্তর প্রায়ই "যেটা বেশি দেখা যায়"। আসল উত্তর: **যেটার উপর অন্য সব দাঁড়িয়ে** — academics। নইলে পরে সব মডেল বদলাতে হবে।

ধাপ ৩

অ্যাপ × ভূমিকা — কে কোন ঘরে ঢোকে

সংখ্যাগুলো ক্যাটালগ থেকে গোনা; খালি ঘর মানে "এই ভূমিকার জন্য কোনো ফিচার নেই"

অ্যাপ	শিক্ষক	অ্যাডমিন	কোঅর্ডি.	শিক্ষার্থী	প্রশাসক	পাবলিক	মোট
accounts	২	১	—	—	—	৪	**৭**
workspaces	১	৪	—	—	২	—	**৭**
core (ড্যাশবোর্ড)	১	—	১	১	১	—	**৪**
academics	৩	৫	—	—	—	—	**৮**
students	৪	—	—	—	—	—	**৪**
attendance	৪	—	১	—	—	—	**৫**
lessons	৬	—	—	১	—	—	**৭**
assessments	৪	১	—	২	—	—	**৭**
reports	৪	—	—	—	—	—	**৪**
audit	—	১	—	—	—	—	**১**
billing + সেটিংস	—	৩	—	—	—	—	**৩**
মোট	**২৯**	**১৫**	**২**	**৪**	**৩**	**৪**	**৫৭**

এই গ্রিডটাই ক্লাস ৯-এর **পারমিশন ম্যাট্রিক্সের** ভিত্তি — শুধু উল্টো করে পড়তে হবে: "এই ঘরে কে আছে" থেকে "এই ঘরে কে থাকবে না"।

শিক্ষকের নোট: ডান কলামের যোগফল ৫৭ — মিলছে কি না জিজ্ঞেস করুন। না মিললে বোঝা যায় কোনো ফিচার দুই ভূমিকায় গোনা হয়েছে, আর সেটাই পরে দ্বিগুণ কাজ হিসেবে ফিরে আসে।

ধাপ ৪

ফ্রিজে সই — তিনটি বাক্স, চারটি শর্ত

সই মানে "থাকবে না" বলা, শুধু "থাকবে" নয়

🧊 MVP — ৫টি স্লাইস

লগইন · প্রতিষ্ঠান ও ভূমিকা · শিক্ষার্থী ও ভর্তি · উপস্থিতি · নম্বর ও রিপোর্ট

≈ ৪৩টি ফিচার

🕐 v1.1

আমন্ত্রণ · ব্যাকআপ · পূর্ণ এক্সপোর্ট · নোটিফিকেশন

৪–৫টি ফিচার

🚫 কখনো নয়

SMS গেটওয়ে · মোবাইল অ্যাপ · AI উপস্থিতি

কারণ লিখে রাখা

চারটি শর্ত — সইয়ের আগে

১. প্রতিটি ফিচারের **আইডি** আছে

২. প্রতিটির **এক বাক্য উদ্দেশ্য** আছে

৩. প্রতিটির **একটি ভূমিকা** আছে (দুই হলে ভাঙুন)

৪. প্রতিটির **একটি রুট** আছে — না থাকলে সেটা ফিচার নয়, ইচ্ছা

স্কোপ ফ্রিজের আসল পরীক্ষা: "কোনটা বাদ দিলে প্রজেক্ট মরবে না?" — এই প্রশ্নের উত্তর দিতে না পারলে ফ্রিজ হয়নি।

শিক্ষকের নোট: স্লাইডে ৪৩ নম্বরটা ইচ্ছাকৃত — এটা স্ক্রিনশট-প্রমাণসহ ফিচারের সংখ্যা, আর ৫৭ মোট পরিকল্পনা। দুটো সংখ্যা গুলিয়ে ফেলা যাবে না; ক্লাস ১৩-এ এই পার্থক্যটাই ডেমোর সততা।

ধাপ ৫

এক ফিচার, এক মালিক

দুই ভূমিকার ফিচার মানে দুইবার পারমিশন যাচাই — তাই ভাঙতে হয়

প্রলোভন	কেন ভাঙতে হয়	প্রজেক্টে যা করা হলো
"উপস্থিতি শিক্ষক ও কোঅর্ডিনেটর — দুজনেই দেখবে"	দেখার নিয়ম দুজনের আলাদা: শিক্ষক নিজের ক্লাস, কোঅর্ডিনেটর পুরো বিভাগ	**F-33** রেজিস্টার (শিক্ষক) আর **F-35** oversight (কোঅর্ডিনেটর) — আলাদা ফিচার
"ফলাফল শিক্ষক ও শিক্ষার্থী — একই পর্দা"	শিক্ষক হিসাব করেন, শিক্ষার্থী শুধু প্রকাশের পরে দেখে	**F-46** হিসাব · **F-47** প্রকাশ · **F-48** আমার ফলাফল
"সেটিংস সবই এক পাতা"	গ্রেডিং আর ব্র্যান্ডিং সম্পূর্ণ আলাদা সিদ্ধান্ত	**F-45** গ্রেড স্কেল · **F-30** প্রতিষ্ঠান ও ব্র্যান্ডিং · **F-55** উপস্থিতি নীতি
"ড্যাশবোর্ড একটাই, শুধু ডেটা বদলাবে"	চার ভূমিকার প্রশ্নই আলাদা — এক পর্দায় চার প্রশ্নের উত্তর হয় না	**F-13** · **F-16** · **F-29** · **F-56** — চারটি ড্যাশবোর্ড

নিয়ম: **এক ফিচার = এক ভূমিকা = এক রুট = এক পরীক্ষা।** দ্বিতীয় ভূমিকা দরকার হলে সেটা নতুন ফিচার, পুরনোতে শর্ত নয়।

শিক্ষকের নোট: তৃতীয় সারিটা সবচেয়ে বেশি কাজে লাগে — "সব সেটিংস এক জায়গায়" শুনতে সুন্দর, কিন্তু তার মানে একজনের হাতে গ্রেডিং আর ব্র্যান্ডিং দুটোই। ছাত্রদের বলুন: সুবিধা আর অধিকার আলাদা জিনিস।

ধাপ ৬

চার ধরনের ফিচার — দাম আলাদা

তালিকা বানানোর সময়ই ধরন লিখে রাখুন, তখন সময়ের হিসাব নিজে থেকে আসবে

তালিকা (list)

প্রশ্ন: **কী আছে?**
অংশ: ভিউ + ১ টেমপ্লেট + ১ টেস্ট
সময়: **আধা দিন**
F-19 ছাত্র তালিকা

ফর্ম (form)

প্রশ্ন: **কী লিখব?**
অংশ: ফর্ম + ভ্যালিডেশন + ৪টি ভুল-অবস্থা
সময়: **১ দিন**
F-31 হাজিরা

রিপোর্ট (report)

প্রশ্ন: **কত?**
অংশ: কোয়েরি + অ্যাগ্রিগেট + এক্সপোর্ট
সময়: **২ দিন**
F-34 সারাংশ

মডেল (model)

প্রশ্ন: **কোথায় রাখব?**
অংশ: মাইগ্রেশন + অ্যাডমিন + পুরনো ডেটার যত্ন
সময়: **৩ দিন**
F-24 শিক্ষাবর্ষ

তালিকায় ৫৭টি লাইন দেখে "১০ দিনে হয়ে যাবে" ভাববেন না — **প্রতি লাইনের ধরন দেখুন**। নয়টি মডেল মানেই তিন সপ্তাহ।

শিক্ষকের নোট: ক্লাসে একটা ছোট কাজ দিন: ৫৭টি ফিচারকে চার ধরনে ভাগ করতে বলুন (মোটামুটি)। যে দলে "মডেল" বেশি পড়বে, তার প্রজেক্টের সময়সূচি সবচেয়ে বেশি ভাঙবে।

ধাপ ৭

ডুপ্লিকেট ধরা — একই জিনিস দুই নামে

তালিকা বড় হয় না, ফুলে যায় — আর ফোলা তালিকা কেউ পড়ে না

দুটো লাইন যেমন দেখায়	আসল প্রশ্ন	সিদ্ধান্ত
"আমার ক্লাস" · "ক্লাস ও সেকশন"	একই টেবিল, ভিন্ন দর্শক?	**দুটোই রাখা** — F-17 শিক্ষকের দেখা, F-26 অ্যাডমিনের তৈরি
"হাজিরা নেওয়া" · "হাজিরা রেজিস্টার"	একই ডেটা, দুই কাজ?	**দুটোই রাখা** — লেখা আর পড়া আলাদা কাজ (F-31 · F-33)
"ফলাফল" · "মার্কশিট PDF"	একটা অন্যটার রূপ?	**আলাদা** — PDF-এর আলাদা রেন্ডারার ও আলাদা টেস্ট (F-48 · F-49)
"সারাংশ" · "ক্লাস সারাংশ"	একই রিপোর্ট?	**এক করে ফেলা** — F-52 এক জায়গায়, ফিল্টার দিয়ে ভাগ

পরীক্ষা: **একই রুট?** → একটাই ফিচার। **একই টেবিল কিন্তু ভিন্ন কাজ?** → দুটো ফিচার। শুধু নাম দেখে মিলিয়ে ফেলা যাবে না।

শিক্ষকের নোট: চতুর্থ সারিটা "এক করে ফেলা"-র উদাহরণ — সব সারি জোড়া লাগাতে হবে না। কখনো এক করলে ফিচার সরল হয়, কখনো দুই ভূমিকা এক হয়ে যায় আর পারমিশন অস্পষ্ট হয়।

ধাপ ৮

যা ফ্রিজে ঢোকেনি — আর কেন

"কখনো নয়" তালিকাটাই আপনাকে সেমিস্টারের মাঝে বাঁচাবে

ফিচার	বাক্স	কারণ (এক লাইনে)	কখন ফিরবে
সদস্য আমন্ত্রণ	**v1.1**	ইমেইল পাঠানোর সেটআপ দরকার; এখন অ্যাডমিন নিজে অ্যাকাউন্ট খুলে দেন	একই কাজ দুই জায়গায় হলে
ব্যাকআপ কনসোল	**v1.1**	সার্ভারে cron + pg_dump যথেষ্ট; UI দরকার নেই	একাধিক প্রতিষ্ঠান এলে
পূর্ণ এক্সপোর্ট (XLSX)	**v1.1**	CSV প্রথমে যথেষ্ট, বাংলা সমস্যা হলে XLSX	গ্রাহক চাইলে
SMS নোটিফিকেশন	**কখনো নয়**	প্রতি SMS-এ টাকা লাগে; খরচ মডেল বদলে যাবে	প্ল্যানে আলাদা মূল্য যোগ হলে
মোবাইল অ্যাপ	**কখনো নয়**	ওয়েবসাইট মোবাইলে কাজ করে; অ্যাপ একই কাজ দুইবার	অফলাইন দরকার হলে
AI উপস্থিতি (ছবি থেকে)	**কখনো নয়**	গোপনীয়তা, নির্ভুলতা, খরচ — তিনটেই বাইরে	ক্যামেরা ও অনুমোদন থাকলে

শিক্ষকের নোট: "কখন ফিরবে" কলামটা বাদ দেবেন না — এটি ঝুঁকি ব্যবস্থাপনা: ফিচার ফিরে আসবে, কিন্তু শর্ত পূরণ হলে। শর্ত ছাড়া ফেরা মানেই স্কোপ ফাটল।

ধাপ ৯

তালিকা কোথায় থাকে — ক্যাটালগ ফাইলের গঠন

একটি ফাইলে ৫৭টি ফিচার, প্রতিটির ছয়টি কী — `docs/catalogue_data.py`

```
FEATURES = [
  dict(id="F-31", role="TEACHER",
       name="Take attendance",
       route="/w/<id>/attendance/take/",
       status="built",
       layout=SUBMIT,
       purpose="Marks a whole class present/
                absent/late/leave for one date",
       actions=["Pick class, date, subject",
                "Bulk buttons",
                "Add a note", "Save"],
       data="AttendanceRecord unique(student,
             class, subject, date)",
       states="Already taken today, future date
               blocked, holiday warning",
       cls="C099, C100"),
  ...
]

ROLES = { "PUBLIC": "Anonymous visitor",
          "TEACHER": "Individual teacher", ... }
```

ছয়টি কী কেন

id — সংযোগের নাম
role — কোন ভূমিকা
route — কোথায় থাকে
purpose — এক বাক্যে কেন
actions — পর্দায় কী কী
data · states — কোন টেবিল, কোন ভুল-অবস্থা

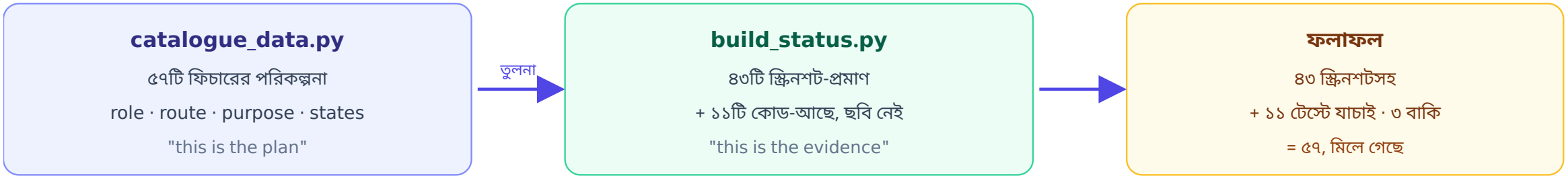
তালিকা **কোডে** রাখার কারণ: তালিকা থেকে ডকুমেন্ট, PDF, স্ক্রিনশট-চেক — সব একই সূত্র থেকে তৈরি হয়।

শিক্ষকের নোট: "Excel-এ তালিকা রাখলেই তো হয়" — এই প্রশ্ন আসবেই। উত্তর: Excel থেকে কোনো কিছু তৈরি করা যায় না; কোড থেকে PDF, চেক, টেস্ট — সবই যায়, আর সেটাই কমিট 5373b59-এ করা হয়েছে।

ধাপ ১০

পরিকল্পনা আর প্রমাণ — দুই ফাইল, ইচ্ছাকৃত

একই ফাইলে রাখলে স্ক্রিনশট বানাতে গিয়ে তালিকা বদলে যেত



ফাইলটির নিজের মন্তব্য: "Regenerating a screenshot list can never corrupt the feature specification."
অর্থাৎ স্ক্রিনশট আবার তুললে পরিকল্পনা বদলাবে না — দুইটা আলাদা ফাইল থাকার এটাই কারণ।

> এটাই ক্লাসের সবচেয়ে ব্যবহারিক পরামর্শ: **পরিকল্পনা আর প্রমাণ কখনো একই ফাইলে রাখবেন না।**

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে দুটো ফাইল বানাতে — `plan.md` আর `done.md`। প্রতি সপ্তাহে `done.md` আপডেট হবে, `plan.md` নয়। ক্লাস ১৩-এর ডেমোতে এই দুটো ফাইলই তাদের সততার প্রমাণ।

ধাপ ১১

ফ্রিজ ভাঙার প্রক্রিয়া — চার লাইনের অনুরোধ

ফ্রিজ মানে পরিবর্তন বন্ধ নয়; ফ্রিজ মানে পরিবর্তন লিখিত

পরিবর্তনের অনুরোধ (CR)

১. **কী:** কোন ফিচার, কোন আইডি

২. **কেন:** কোন ব্যবহারকারীর কোন বাক্য

৩. **কী বদলাবে:** কোন ফিচার বাদ যাবে বা পিছাবে

৪. **কে অনুমোদন:** নাম ও তারিখ

নিয়ম: শুধু যোগ করা যায় না — কিছু বদলাতে হবে। স্কোপ বাড়ে না, শুধু অদল-বদল হয়।

আসল উদাহরণ — কমার্শিয়াল লেয়ার

কী: সাবস্ক্রিপশন, ট্রায়াল, হোয়াইট-লেবেল, প্ল্যাটফর্ম কনসোল

কেন: "এক স্কুলের সফটওয়্যার নয়, প্ল্যাটফর্ম" — ক্লাস ১-এর সিদ্ধান্ত

কী বদলালো: সাবস্ক্রিপশন ব্যক্তি থেকে **প্রতিষ্ঠানে** (`8a25724`)

খরচ: ১ মাইগ্রেশন + বিলিং ভিউ + ৪ টেস্ট

শিক্ষকের নোট: এই স্লাইডে সততার প্রশ্ন তুলুন: প্রজেক্টেও স্কোপ বেড়েছিল — কমার্শিয়াল লেয়ারটা ফ্রিজের বাইরে ছিল। বলুন: "আমরাও ভুল করেছি; পার্থক্য একটাই — ভুলটা লিখে রাখা হয়েছে।"

আলোচনা

চারটি প্রশ্ন — নিজের তালিকা মিলিয়ে

প্রতিটি উত্তর একটা সংখ্যা দিয়ে শেষ করুন

১. আপনার প্রজেক্টে এখন কতটি ফিচার? সংখ্যাটা লিখুন — ২০-এর বেশি হলে ভাগ হয়নি, ফুলেছে।

২. কোন ফিচারের **দুটো ভূমিকা** আছে? সেটা ভাঙলে কতটি হবে?

৩. আপনার "কখনো নয়" বাক্সে কয়টি ফিচার, আর প্রতিটির কারণ এক লাইনে লিখতে পারবেন?

৪. আপনার তালিকার কোন লাইনের **রুট নেই**? রুট না থাকলে সেটা ফিচার নয় — ইচ্ছা।

শিক্ষকের নোট: ৪ নম্বরের উত্তর দিয়ে ক্লাস ৭ শুরু হবে — 루트 없는 লাইনগুলোই পরে "এটা তো হয়নি" বলে ধরা পড়ে, আর ডেমোর দিনে সবচেয়ে বিব্রতকর হয়।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
ফিচার মানে আইডি + উদ্দেশ্য + রুট + ভূমিকা	F-33-এর ছয়টি কী হুবহু `catalogue_data.py` থেকে
এক ফিচার = এক ভূমিকা — দ্বিতীয় ভূমিকা মানে নতুন ফিচার	উপস্থিতি ভাঙা হয়েছে: F-33 (শিক্ষক) · F-35 (কোঅর্ডিনেটর)
তালিকার দাম তার ধরনে — list · form · report · model	৯টি মডেল-ধরনের ফিচার = প্রায় ৩ সপ্তাহের কাজ
"কখনো নয়" বাক্স + কারণ — সবচেয়ে দরকারি অংশ	SMS · অ্যাপ · AI — তিনটিরই কারণ ও ফেরার শর্ত লেখা
পরিকল্পনা আর প্রমাণ আলাদা ফাইল	`catalogue_data.py` বনাম `build_status.py` → ৪৩ + ১১ + ৩ = ৫৭

শিক্ষকের নোট: পাঁচটি লাইন একসাথে পড়ে দেখুন — এগুলোই পর্ব ৪-এর প্রতিটি স্লাইসের শিরোনাম হবে। ফিচার আইডি ছাড়া স্লাইস বানানো যায় না, কারণ স্লাইসের সীমা ফিচার দিয়েই টানা হয়।

০৭

পরের ক্লাস — "কেন এই স্ট্যাক"

তিনটি বিকল্পের তুলনা · ADR লেখার নিয়ম · সিদ্ধান্ত লিখে রাখা

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৭ / ৩৭

কেন এই স্ট্যাক — আর কেন নয়

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.৩ (প্রযুক্তি নির্বাচন ও ADR)
তিন বিকল্পের তুলনা · সিদ্ধান্তের নথি · সিদ্ধান্ত কোথায় থাকে

৬টি প্যাকেজ · ৩টি settings · ১টি ADR-নিয়ম

শুরু করুন প্রশ্ন দিয়ে

"কেন Django?" — এই প্রশ্নের উত্তর মুখস্থ নয়, লিখিত হতে হবে

হাত তুলুন: আপনার স্ট্যাক কেন বেছেছেন — এক বাক্যে বলুন

"সবাই ব্যবহার করে" — এটা কারণ নয়।

কারণ হলো: এই কাজে এটার কী সুবিধা, আর কী দাম।

প্রযুক্তি বাছাই দুটি প্রশ্নের উত্তর: **"কী সহজ হবে?"** আর **"কী কঠিন হবে?"**। দুটোর উত্তর লিখে রাখলে সেটাকে বলে **ADR** — Architecture Decision Record।
না লিখলে ছয় মাস পরে আপনি নিজেই জানবেন না কেন এটা বেছেছিলেন।

শিক্ষকের নোট: কেউ "React শিখতে চাই বলে React" বললে খুব ভালো — কিন্তু সেটাও লিখে রাখতে বলুন। ADR-এ "আমি এটা শিখতে চাই" একটি সম্পূর্ণ বৈধ কারণ, যদি সেটা লেখা থাকে।

ধাপ ১

স্ট্যাকের ছয়টি স্তর — প্রতিটির একটা সিদ্ধান্ত

"স্ট্যাক" মানে শুধু ফ্রেমওয়ার্ক নয় — ছয় জায়গায় ছয়টা পছন্দ

১. ভাষা
Python 3.13 — কোর্সের ভাষা

২. ফ্রেমওয়ার্ক
Django 6.1.2 — auth · ORM · forms

৩. ডেটাবেস
PostgreSQL · sqlite (ডেমো)

৪. পর্দা
Django templates · ৭৫টি — SPA নয়

৫. ফাইল আউটপুট
openpyxl · reportlab — Excel · PDF

৬. সার্ভার
gunicorn · ২ worker

নিয়ম: প্রতিটি স্তরের পছন্দের জন্য একটি করে ADR — ছয়টি স্তর, ছয়টি সিদ্ধান্ত, ছয়টি কারণ।
এগুলোর কোনোটাই "ডিফল্ট" নয় — প্রতিটি বেছে নেওয়া, আর প্রতিটির একটা না-বাছাইয়ের বিকল্প ছিল।

শিক্ষকের নোট: ছয়টি বাক্স বোর্ডে আঁকুন। ছাত্ররা সাধারণত ২ আর ৩ নিয়ে ভাবে, ৪ ও ৬ ভুলে যায় — অথচ ডেমোর দিনে সমস্যা হয় ঠিক ওই দুটোতেই।

ধাপ ২

তিনটি বিকল্প — একই কাজ, তিন পথ

স্কোর নয়, **প্রতিদান** — প্রতিটি সারিতে "কী পাব, কী হারাব"

মানদণ্ড	Django (Python)	Node + Express + React	Laravel (PHP)
লগইন ও ভূমিকা	✓ বিল্ট-ইন, ২ ঘণ্টার কাজ	✗ নিজে: JWT, রিফ্রেশ, গার্ড	✓ বিল্ট-ইন
ডেটাবেস বদল	✓ মাইগ্রেশন স্বয়ংক্রিয়	◐ Prisma/Sequelize যোগ	✓ Eloquent মাইগ্রেশন
৩১ টেবিল, ৮টি সম্পর্ক	✓ ORM সহজে সামলায়	◐ সম্ভব, কাঠামো নিজের	✓ সম্ভব
পর্দা বানানো	✓ টেমপ্লেট — ১ ফাইলে ১ পর্দা	✗ বিল্ড স্টেপ, npm, bundler	✓ Blade
এক কমান্ডে চালু	✓ `run.sh`	✗ ২টি সার্ভার + ২টি পোর্ট	◐ সম্ভব
কোর্সের ভাষা	✓ Python	✗ JavaScript	✗ PHP
ডাকুমেন্টেশন	✓ সর্বোত্তম	✓ ভালো	✓ ভালো

খেয়াল করুন: **React-এর কোনো সারিতে "খারাপ" লেখা নেই** — শুধু "এই প্রজেক্টে অতিরিক্ত"। ADR-এ এভাবে সৎ ভাষা লিখতে হয়।

শিক্ষকের নোট: কেউ "React বাদ কেন" জিজ্ঞেস করলে বলুন — "বাদ নয়, দামি।" ৩১ টেবিলের CRUD-এ SPA-র সুবিধা কম, খরচ বেশি। ছাত্রদের নিজের প্রজেক্টে একই টেবিল লিখতে বলুন।

ধাপ ৩

কেন SPA নয় — সিদ্ধান্তটার সৎ হিসাব

৭৫টি টেমপ্লেট · কোনো npm · কোনো বিল্ড স্টেপ নেই

সার্ভার-রেন্ডারড যা দেয়

- ✓ প্রথম পাতাই পূর্ণ HTML — ধীর নেটেও পড়া যায়
- ✓ SEO নিজে থেকেই (পাবলিক সাইট)
- ✓ এক সার্ভার, এক পোর্ট, এক কমান্ড
- ✓ ফর্ম ভ্যালিডেশন সার্ভারে — বাইপাস করা যায় না
- ✓ ৪ ভূমিকার জন্য ৪টি টেমপ্লেট — শর্ত দিয়ে লুকানো নয়

যা হারাতে হয়েছে

- ✗ পূর্ণ-পাতা রিলোড — একটু ধীর অনুভূত হয়
- ✗ অফলাইনে কাজ করে না
- ✗ জটিল ইন্টারঅ্যাকশন (টানা ড্র্যাগ) কষ্টকর
- ✗ BFF/রিয়েল-টাইম পরে যোগ করতে হবে

সিদ্ধান্ত: ৪৩টি ফিচারের **৩৮টিই ফর্ম-ভিত্তিক** — অর্থাৎ SPA-র সুবিধা প্রায় লাগবেই না। তাই সার্ভার-রেন্ডারডই সঠিক পছন্দ।

শিক্ষকের নোট: এখানে সততার একটা উদাহরণ দিন: "যদি আপনার প্রজেক্টে লাইভ চ্যাট বা ক্যানভাস অঙ্কন থাকে, তাহলে এই সিদ্ধান্ত উল্টো হবে।" ADR নির্দিষ্ট প্রজেক্টের জন্য, সব প্রজেক্টের জন্য নয়।

ধাপ ৪

ছয়টি প্যাকেজ, ভার্সন পিন করা — হুবহু ফাইল থেকে

`requirements.txt`-এর নিজের মন্তব্য: "Exact versions used and verified in this build (2026-10-29, Python 3.13)"

```
Django==6.1.2
psycopg[binary]==3.3.6
python-dotenv==1.2.4
Pillow==12.3.0

# Used for exports and the marksheet PDF
openpyxl==3.1.5
reportlab==5.1.0
```

== কেন, >= কেন নয়

>= লিখলে আজ ঠিক চললেও ছয় মাস পরে নতুন ভার্সন আসবে — আর আপনার কোড ভাঙতে পারে।

== লিখলে "আজ যা চলছে, সেটাই চলবে"।

আপডেট তখনই, যখন **ইচ্ছাকৃতভাবে** করবেন — টেস্ট চালিয়ে, এক প্যাকেজ করে।

৬টি প্যাকেজই যথেষ্ট — ৩১ টেবিল, ৭৫ পর্দা, PDF, Excel, PostgreSQL। লাইব্রেরির সংখ্যা কম মানে দুর্বলতা নয়, নিয়ন্ত্রণ।

শিক্ষকের নোট: "pip freeze" করে ফাইলটা বানানোর কথা বলুন — হাতে লেখা নয়। আর commit করার আগে ভার্সনগুলো মিলিয়ে নেওয়া, নইলে ডেমোর দিনে ভিন্ন মেশিনে ভিন্ন ভার্সন চলবে।

ধাপ ৫

ADR কী — এক পাতার সিদ্ধান্ত-নথি

পাঁচটি অংশ, একটির বেশি নয় — বড় হলে সিদ্ধান্তটা এখনো স্পষ্ট নয়

১. প্রসঙ্গ

কী পরিস্থিতিতে
সিদ্ধান্ত নিতে হচ্ছে

৩–৪ লাইন

২. সিদ্ধান্ত

এক বাক্যে
"আমরা X ব্যবহার করব"

আপস নেই

৩. বিকল্প

অন্তত দুটি
+ কেন বাদ

এটাই ADR-এর প্রাণ

৪. পরিণাম

কী লাভ, কী দাম
কী কঠিন হয়ে যাবে

সৎ ভাষায়

৫. তারিখ ও নাম

কবে, কে
কোন ক্লাসে

পরে খুঁজে পাওয়ার জন্য

পাঁচটি ঘর ভরে ফেলুন — এক পাতায়, বাংলায়, নিজের ভাষায়। এক ঘর খালি থাকলে সিদ্ধান্তটা অসম্পূর্ণ।

ADR মূলত ভবিষ্যতের নিজের কাছে লেখা চিঠি — "কেন এটা করেছিলাম" তার উত্তর।

শিক্ষকের নোট: "৪ নম্বর ঘরটা লিখতে গিয়ে থেমে যাবেন — সেটাই আসল কাজ।" যে সিদ্ধান্তের কোনো খারাপ দিক নেই, সেটা সিদ্ধান্ত নয়, ইচ্ছা।

ADR-001

ভরা উদাহরণ — কেন Django

পাঁচটি ঘর, পাঁচটি উত্তর — এক পাতায়

ADR-001 · ফ্রেমওয়ার্ক নির্বাচন

২০২৬-১০-২৯ · ক্লাস C071

প্রসঙ্গ: ৩১ টেবিল, ৬ ভূমিকা, ৫৭ ফিচারের একটি প্রতিষ্ঠান-ব্যবস্থাপনা সিস্টেম; টিম এক জন; সময় ১২ সপ্তাহ। লগইন, পারমিশন ও অডিট — তিনটেই লাগবেই।

সিদ্ধান্ত: Django 6.1.2 (Python 3.13), সার্ভার-রেন্ডারড টেমপ্লেট সহ। কোনো SPA ফ্রেমওয়ার্ক নয়।

বিকল্প ও কেন বাদ: **Flask** — auth/ভূমিকা নিজে লিখতে হতো (+~২০০০ লাইন); **Node + React** — দুই সার্ভার, বিল্ড স্টেপ, আর কোর্স Python-ভিত্তিক; **Laravel** — সমমানের ভালো, কিন্তু ভাষা PHP; **No-code** — কোড শেখার উদ্দেশ্যই ভাঙে।

পরিণাম: লাভ — ২ ঘণ্টায় লগইন+ভূমিকা, স্বয়ংক্রিয় মাইগ্রেশন, admin প্যানেল। দাম — Django-র শেখার বাঁক, আর জটিল ইন্টারঅ্যাকশনে কম স্বাধীনতা।

শিক্ষকের নোট: এই পাতা হুবহু ছাত্রদের খাতায় লিখিয়ে নিন — তারপর নিজের প্রজেক্টের জন্য একই পাঁচ ঘর ভরতে বলুন। ভাষা বদলাবে, কাঠামো একই থাকবে।

ADR-002

যে সিদ্ধান্তটা একবারই নেওয়া যায় — কাস্টম User

কোডের নিজের মন্তব্য: "never start a Django project with the default User model"

ADR-002 · কাস্টম ইউজার মডেল

প্রসঙ্গ: প্রথম মাইগ্রেশনের আগে ঠিক করতে হবে — ইউজারকে ইমেইল দিয়ে লগইন করাব? ফোন? নাম আলাদা রাখব?

সিদ্ধান্ত: ইমেইল-ভিত্তিক কাস্টম `User` মডেল, প্রথম মাইগ্রেশনের **আগে**।

বিকল্প: ডিফল্ট `User` (username দিয়ে) — পরে বদলানো প্রায় অসম্ভব।

পরিণাম: লাভ — ইমেইল লগইন, ভূমিকা আলাদা অ্যাপে। দাম — নিজে ম্যানেজার লিখতে হয়েছে।

কেন এটা "একবারই"?

Django-তে ইউজার মডেল বদলাতে হলে **সব মাইগ্রেশন নতুন করে** করতে হয় — অর্থাৎ ডেটাবেস খালি করা। স্কুলের এক বছরের উপস্থিতি থাকলে সেটা চলে না।

সিদ্ধান্তের চিহ্ন কোডে

```
AUTH_USER_MODEL = "accounts.User"
# MUST be set before the first migration
```

শিক্ষকের নোট: এই ADR-টা ছাত্রদের সবার আগে লিখতে বলুন — এটাই একমাত্র সিদ্ধান্ত যা ভুল হলে পুরো কাজ নতুন করে শুরু করতে হয়।

ADR-003

যে সিদ্ধান্ত ভুল হয়েছিল — টাকার একক

এটাই ADR লেখার সবচেয়ে বড় যুক্তি: ভুলটাও লিখে রাখতে হয়

প্রথম সিদ্ধান্ত (ভুল)

সাবস্ক্রিপশন **ব্যবহারকারীর** নামে বাঁধা। কারণ তখন ধরা হয়েছিল "একজন মানুষই একাউন্ট খোলে।"

ফল: শিক্ষক দুই স্কুলে পড়ালে বিল কার নামে? প্রতিষ্ঠানের সিট কে ব্যবহার করছে — সেই প্রশ্নের উত্তরই নেই।

ADR-003 (সংশোধিত · `8a25724`)

সিদ্ধান্ত: সাবস্ক্রিপশন **Workspace-এ**; মানুষ শুধু সদস্য।
পরিণাম: বিল প্রতিষ্ঠানের, সিট প্রতিষ্ঠানের; শিক্ষক একাধিক প্রতিষ্ঠানে থাকতে পারেন।
দাম: ১টি মাইগ্রেশন + বিলিং ভিউ + ৪টি টেস্ট।

নিয়ম: পুরনো ADR **মুছবেন না** — "superseded by ADR-003b" লিখে রাখুন। ভুলের ইতিহাসই সবচেয়ে দামি নথি।

শিক্ষকের নোট: শিক্ষকেরা জিজ্ঞেস করবেন "ভুল করা ADR কেন রাখব?" উত্তর: কারণ তিন মাস পরে কেউ একই ভুল করতে যাবে। মুছে ফেলা মানে তাকে একই দেয়ালে আবার ধাক্কা খেতে দেওয়া।

ধাপ ৬

সিদ্ধান্ত কোথায় থাকে — প্রজেক্টের আসল প্রমাণ

`docs/`-এ ADR ফোল্ডার নেই; কিন্তু সিদ্ধান্তগুলো আছে ফাইলের মন্তব্যে

ফাইল	যা লেখা আছে (হুবহু)	কোন সিদ্ধান্ত
`config/settings/base.py`	"Nothing secret lives here. Everything that changes between dev, demo and production is read from environment variables."	সেটিংস ভাগ
`config/settings/prod.py`	"Every value here is deliberate. Nothing in this file may be relaxed to make a deployment 'work'."	প্রোডাকশন কঠোরতা
`config/settings/sqlite.py`	"Use this ONLY to try the application quickly… The real stack (PostgreSQL) remains the documented path."	ডেমো ডেটাবেস
`apps/accounts/models.py`	"Custom user model — created BEFORE the first migration… never start a Django project with the default User model."	কাস্টম User
`docs/build_status.py`	"the catalogue is the *plan*, this file is the *evidence*."	পরিকল্পনা/প্রমাণ পৃথক
`apps/accounts/urls.py`	পাসওয়ার্ড রিসেটের চারটি রুট একসাথে	রিসেট ফ্লো

এগুলো ADR নয়, কিন্তু **ADR-এর কাজ করছে** — কারণ প্রতিটি ভবিষ্যতের পাঠককে "কেন" বলছে।

শিক্ষকের নোট: ভালো অভ্যাস: নিজের প্রজেক্টে `docs/adr/` ফোল্ডার বানিয়ে `001-...md` রাখুন। প্রজেক্টে যেটা হয়নি, সেটা ছাত্ররা আরও ভালোভাবে করতে পারে — বলুন এটা একটা উন্নতির সুযোগ।

ধাপ ৭

ADR লেখার ছয়টি নিয়ম

এক পাতা না হলে সিদ্ধান্তটা তখনো স্পষ্ট নয়

১. এক ADR = এক সিদ্ধান্ত

"ডেটাবেস ও টেমপ্লেট" একসাথে লিখবেন না — পরে একটা বদলালে অন্যটার ইতিহাস হারায়।

২. বিকল্প অন্তত দুটি

একটাই বিকল্প মানে সিদ্ধান্ত নয়, অনুমোদন। যে দুটি বিকল্প ভেবেছেন, দুটিই লিখুন।

৩. পরিণামে খারাপ দিকও

"কোনো অসুবিধা নেই" — এই বাক্য থাকলে ADR-টা বিশ্বাসযোগ্য নয়।

৪. তারিখ ও নাম

কে, কবে, কোন ক্লাসে — পরে "এটা কার ধারণা ছিল" প্রশ্নের উত্তর থাকবে।

৫. বদলালে নতুন ADR

পুরনোটা মুছবেন না; নতুনটায় "supersedes ADR-003" লিখুন।

৬. কোড দেখিয়ে লিখুন

"আমরা X ব্যবহার করি" নয় — **কোন ফাইলে, কোন লাইনে** সেটা পাওয়া যায়।

দৈর্ঘ্য: **এক পাতা, সর্বোচ্চ দুই পাতা।** তিন পাতায় গেলে সেটা ADR নয়, ডিজাইন ডকুমেন্ট — আলাদা জিনিস।

শিক্ষকের নোট: ৬ নম্বর নিয়মটা সবচেয়ে বেশি বাদ পড়ে — অথচ এটাই ADR-কে অনুমান থেকে প্রমাণে নিয়ে যায়। ছাত্রদের প্রতিটি ADR-এ অন্তত একটি ফাইলের নাম লিখতে বাধ্য করুন।

ধাপ ৮

প্রতিটি পছন্দের একটা দাম আছে

ADR-এর "পরিণাম" ঘরটা সৎভাবে ভরার জন্য

পছন্দ	যা পাওয়া গেল	যা কিনতে হলো (দাম)
Django 6.1.2	auth, ORM, forms, admin — বিনা লেখায়	মেজর আপডেটে ভাঙার ঝুঁকি; শেখার বাঁক
PostgreSQL	৭২ লাখ সারি, ইনডেক্স, লক	সার্ভারে আলাদা ডেটাবেস সার্ভিস চালাতে হয়
sqlite ডেমো	এক ক্লিকে চালু, কোনো সেটআপ নেই	দুই পথ — ডেমো আর প্রোডাকশনে আচরণ আলাদা হতে পারে
সার্ভার-রেন্ডারড	এক সার্ভার, SEO, সহজ ডিপ্লয়	পূর্ণ-পাতা রিলোড; সমৃদ্ধ ইন্টারঅ্যাকশন কষ্টকর
reportlab + openpyxl	মার্কশিট PDF, Excel এক্সপোর্ট	দুই লাইব্রেরি, দুই ধরনের টেস্ট
gunicorn ২ worker	সহজ, কম মেমরি	এক worker মরলে অর্ধেক ক্ষমতা; পরে স্কেলিং লাগবে

শিক্ষকের নোট: ছাত্রদের বলুন নিজের স্ট্যাকের জন্য এই তিন-কলামের টেবিল বানাতে — বিশেষ করে ডান কলামটা। যে টেবিলে ডান কলাম খালি, সে টেবিলটা মিথ্যা বলছে।

সতর্কতা

বাংলা + PDF-এর ফাঁদ — আমাদের নিজের অভিজ্ঞতা

এই ডেকটাই যেভাবে তৈরি হয়েছে, সেটাই এখানে প্রমাণ

যা কাজ করে না

reportlab বাংলা অক্ষর জোড়া লাগাতে পারে না — "ক্ষ" আলাদা দুই অক্ষর হয়ে যায়, মাত্রা (vowel sign) জায়গায় বসে না। ফল: PDF-এ অক্ষর দেখতে পাওয়া যায়, কিন্তু পড়া যায় না।

যা কাজ করে

HTML + CSS দিয়ে পাতা সাজিয়ে **হেডলেস ব্রাউজার** দিয়ে PDF বানানো — ব্রাউজারের শেপিং ইঞ্জিন বাংলা নিখুঁতভাবে সামলায়।

এই ডেকের আসল পাইপলাইন

১. বাংলা ফন্ট `@font-face` দিয়ে এমবেড
২. ligature বন্ধ — `font-variant-ligatures:none`
৩. প্রতিটি স্লাইড এক পাতা (338.7 × 190.5 mm)
৪. Chromium: `page.pdf(print_background=True)`
৫. প্রতিটি পাতা ছবি করে চোখে দেখা

শিক্ষা: "লাইব্রেরি আছে" মানে "লাইব্রেরি আপনার ভাষা পারে" নয় — সবসময় নিজের ভাষায় এক পাতা বানিয়ে দেখুন।

শিক্ষকের নোট: এই স্লাইডটা প্রজেক্টের নিজের কষ্ট থেকে — এই পুরো কোর্স কিটই এই পাইপলাইনে তৈরি। ছাত্রদের বলুন: "তোমাদের ডেমোতে বাংলা PDF লাগলে এই পাঁচ ধাপ লাগবেই।"

ধাপ ৯

কোন সিদ্ধান্ত বদলানো সবচেয়ে কঠিন

আগে যেগুলো ঠিক করবেন, সেগুলোই এই তালিকার উপরের দিকে

১. ইউজার মডেল	সব মাইগ্রেশন নতুন করে — ডেটা খালি করা ছাড়া উপায় নেই	**প্রায় অসম্ভব**
২. টেন্যান্টের একক	কে মালিক — ব্যক্তি না প্রতিষ্ঠান? টেবিল, পারমিশন, বিল সব বদলায়	**খুব কঠিন**
৩. ডেটাবেস ইঞ্জিন	dump → restore, SQL ভিন্নতা, দুটি সেটিংস রক্ষণাবেক্ষণ	**কঠিন**
৪. পর্দার প্রযুক্তি	টেমপ্লেট → SPA: প্রতিটি ৭৫ পর্দা আবার লেখা	**খরচসাপেক্ষ**
৫. লাইব্রেরি	একটি প্যাকেজ বদলানো — সাধারণত সহজ, টেস্ট থাকলে	**সহজ**

তাই নিয়ম: **উপরের তিনটি সিদ্ধান্ত প্রথম সপ্তাহে, ADR-সহ।** নিচের দুটি যেকোনো সময়।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "তোমার কোন সিদ্ধান্ত এখনো নেওয়া হয়নি, যেটা তালিকার ১ বা ২ নম্বরে?" বেশিরভাগের ক্ষেত্রে উত্তর থাকবে "টেন্যান্টের একক" — কারণ তারা এটা নিয়ে ভাবেইনি।

ধাপ ১০

ADR আর কমিট — একই সিদ্ধান্তের দুই চিহ্ন

প্রতিটি বড় সিদ্ধান্তের পাশে একটা কমিট থাকা উচিত, আর উল্টোটাও

কমিট	যা ঘটল	কোন সিদ্ধান্ত	কীভাবে যাচাই করবেন
`05c63dc`	স্লাইস ১: কাস্টম ইউজার, লগইন, workspace, ভূমিকা, ১৩ টেস্ট	**ADR-002** কাস্টম User	`git show 05c63dc --stat`
`674375b`	তিনটি settings, `run.sh`, pinned requirements	**ADR-004** পরিবেশ ভাগ	`ls config/settings/`
`7802a82`	কমার্শিয়াল লেয়ার: সাবস্ক্রিপশন, ট্রায়াল, প্ল্যাটফর্ম কনসোল	**ভুল সিদ্ধান্ত** — টাকার একক ব্যক্তি	ওই কমিটের মাইগ্রেশন দেখুন
`8a25724`	"the subscription belongs to the institution, not to individuals"	**ADR-003b** সংশোধন	`git show 8a25724`
`bfd058e`	TMS → IMS, কিন্তু **settings থেকে চালিত**	**ADR-005** নাম কোডে নয়	`grep PRODUCT_NAME config/settings/base.py`

পরীক্ষা: **ADR-এ লেখা প্রতিটি দাবির পাশে একটা কমিট দেখাতে পারবেন?** না পারলে দাবিটা এখনো প্রমাণ হয়নি — শুধু লেখা হয়েছে।

শিক্ষকের নোট: ক্লাস ২৩-এ গিট নিয়ে পুরো ক্লাস আছে; এখানে শুধু সংযোগটা দেখান। ছাত্রদের বলুন প্রতিটি ADR-এ কমিট হ্যাশ লেখার অভ্যাস করতে — এক লাইনে ভবিষ্যতের অনেক প্রশ্নের উত্তর হয়ে যায়।

ক্লাসের কাজ

একটি ADR লিখুন — ১৫ মিনিট, এক পাতা

নিজের প্রজেক্টের সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্তটা নিয়ে

১. সিদ্ধান্ত বাছুন

ভাষা · ফ্রেমওয়ার্ক · ডেটাবেস · টেন্যান্টের একক — একটাই বেছে নিন।

২. প্রসঙ্গ তিন লাইনে

কী বানাচ্ছেন, কতজন ব্যবহার করবে, কত সময় আছে।

৩. দুটি বিকল্প লিখুন

যেটা আপনি সত্যিই ভেবেছিলেন — বানানো বিকল্প নয়।

৪. পরিণামে দাম লিখুন

অন্তত একটি খারাপ দিক। না থাকলে আরেকবার ভাবুন।

৫. ফাইলের নাম দিন

`docs/adr/001-framework.md` — তারিখ ও নিজের নাম সহ।

যাচাই: পাশের জনকে দিন — সে কি ৩০ সেকেন্ডে বলতে পারবে **কী সিদ্ধান্ত, কেন, আর কী দাম**? না পারলে ADR-টা এখনো শেষ হয়নি।

শিক্ষকের নোট: ক্লাসের শেষে ২–৩টি ADR পড়ে শোনান। যেটায় "পরিণাম"-এর ঘর সবচেয়ে সৎ, সেটাকেই সবচেয়ে ভালো ADR বলুন — সবচেয়ে বড় সিদ্ধান্তটা নয়।

আলোচনা

চারটি প্রশ্ন — স্ট্যাক নিয়ে

উত্তর লিখে রাখুন, ক্লাস ৮-এ ফিরে আসবে

১. আপনার স্ট্যাকের **তিনটি** পছন্দ লিখুন — প্রতিটির একটি সুবিধা ও একটি দাম।

২. আপনার কোথায় **দুই ভাষা** লাগছে (Python + JavaScript)? সেটা কি সত্যিই দরকার?

৩. আপনার প্রজেক্টে **PDF বা Excel** আউটপুট লাগবে? কোন লাইব্রেরি, আর বাংলা পারবে কি?

৪. আপনার কোন সিদ্ধান্তটা তালিকার **১ বা ২ নম্বরে**, আর সেটা কি এখনো নেওয়া হয়নি?

শিক্ষকের নোট: ৩ নম্বর প্রশ্নটা হালকা মনে হলেও অনেক প্রজেক্টের ডেমো এখানেই আটকায় — বিশেষ করে বাংলা মার্কশিট। যার উত্তর "পরে দেখব", তার ডেমোর শেষ সপ্তাহটা নষ্ট হবে।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
স্ট্যাক = ছয়টি স্তর, ছয়টি সিদ্ধান্ত — শুধু ফ্রেমওয়ার্ক নয়	ভাষা · ফ্রেমওয়ার্ক · ডেটাবেস · পর্দা · ফাইল · সার্ভার
প্রযুক্তি বাছাই হয় প্রতিদান দিয়ে, স্কোর দিয়ে নয়	Django ২ ঘণ্টায় auth · Node-এ দুই সার্ভার · Laravel সমান কিন্তু ভাষা PHP
ভার্সন পিন করা মানে নিয়ন্ত্রণ	`requirements.txt`: ৬টি প্যাকেজ, সব `==` সহ
ADR = ৫ ঘরের এক পাতা, আর খারাপ দিক লেখা বাধ্যতামূলক	তিনটি ভরা ADR — Django · কাস্টম User · টাকার একক
ভুল সিদ্ধান্তও লিখে রাখুন — মুছবেন না	সাবস্ক্রিপশন সংশোধন `8a25724` · ১ মাইগ্রেশন + ৪ টেস্ট

শিক্ষকের নোট: শেষ লাইনটা সবচেয়ে গুরুত্বপূর্ণ নৈতিক শিক্ষা — ডেমোতে "কোন ভুল করেছিলেন?" প্রশ্নের সেরা উত্তর হলো একটা ADR, কারণ সেটা প্রমাণ করে আপনি ভুল ধরতে পারেন।

০৮

পরের ক্লাস — "১১টি অ্যাপ, এক সিস্টেম"

আর্কিটেকচার: অ্যাপ নির্ভরতার গ্রাফ · স্তর ভাগ · কেন এত অ্যাপ

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৮ / ৩৭

১১টি অ্যাপ, এক সিস্টেম

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.৪ (সিস্টেম আর্কিটেকচার)

অ্যাপ নির্ভরতার গ্রাফ · স্তর ভাগ · চক্র ভাঙা

১১ অ্যাপ · ৫,৩৮০ লাইন · ১১ urls.py

শুরু করুন প্রশ্ন দিয়ে

সব কোড এক ফাইলে রাখলে কী হয়?

হাত তুলুন: কেউ কি এক ফাইলে ৫ হাজার লাইন লিখেছেন?

প্রথম সপ্তাহে ঠিকই চলে।

তৃতীয় সপ্তাহে খোঁজা বন্ধ হয়ে যায়।

আর্কিটেকচার মানে সাজানো নয়, **দায়িত্ব ভাগ করা**। অ্যাপ মানে এক ফোল্ডার = এক দায়িত্ব — উপস্থিতি কোথায় থাকে, তার উত্তর যেন একটা শব্দে দেওয়া যায়: **attendance**।
আর সবচেয়ে বড় লাভ: **পরীক্ষা করা যায়** — একটা অ্যাপের টেস্ট বাকিগুলো না জেনেও চলে।

শিক্ষকের নোট: কেউ "আমার প্রজেক্টে ৫টা ফাইল" বললে জিজ্ঞেস করুন — "উপস্থিতির কোড কোন ফাইলে?" উত্তর যদি হয় "views.py-তে সব আছে", তাহলে আজকের ক্লাসটা তার জন্য সবচেয়ে বেশি কাজে লাগবে।

ধাপ ১

একটি অ্যাপের ভেতরে কী থাকে

চারটি বাধ্যতামূলক ফাইল, তিনটি ঐচ্ছিক — প্রতিটির একটাই কাজ

```
apps/attendance/
  models.py      ← ডেটা: কোন টেবিল
  views.py       ← কাজ: কী হবে
  urls.py        ← ঠিকানা: কোন রুট
  tests.py       ← প্রমাণ: কী যাচাই
  forms.py       ← ইনপুট যাচাই
  services.py    ← জটিল নিয়ম
  migrations/    ← ডেটাবেসের ইতিহাস
```

প্রতিটির এক লাইনের দায়িত্ব

models — ডেটার আকৃতি; এখানে কোনো HTTP নেই

views — অনুরোধ নেওয়া, উত্তর দেওয়া

urls — কোন পথ কোন কাজে

tests — "এটা ঠিক আছে" বলার প্রমাণ

forms — ব্যবহারকারীর ইনপুট ছাঁকা

services — যে নিয়ম view-এ আঁটে না

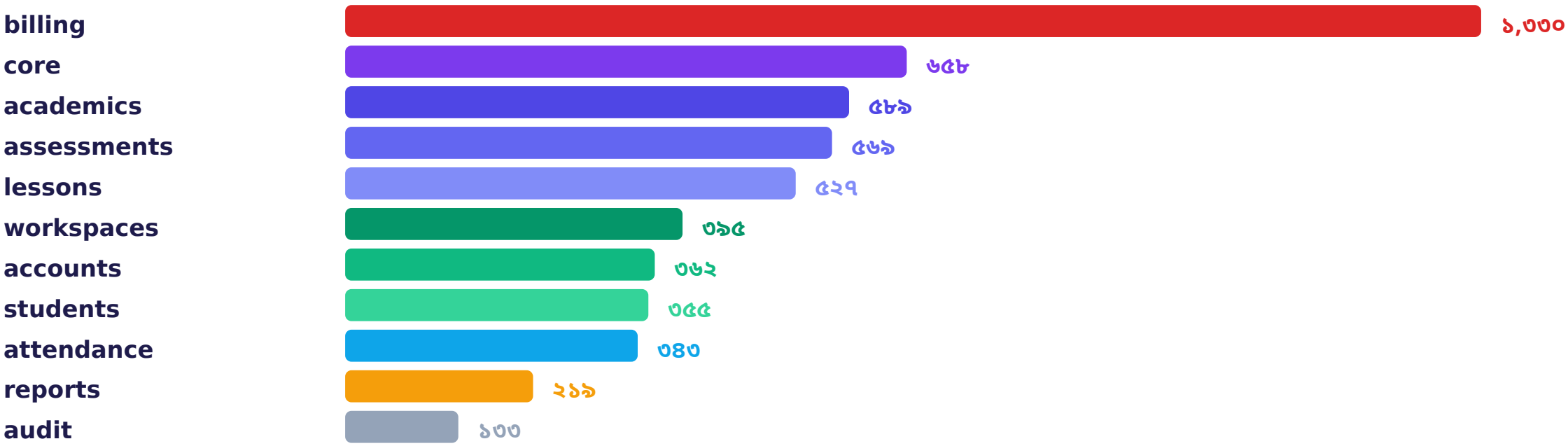
গন্ধ: models.py-তে `HttpResponse` বা views.py-তে ২০০ লাইনের হিসাব — দুটোই দায়িত্ব মিশে যাওয়ার চিহ্ন।

শিক্ষকের নোট: services.py-র উদাহরণ দিন — `apps/billing/services.py`-তে `seat_block_reason` ফাংশন আছে, যেটা view-এ আঁটে না। এই ভাগটা পরে ক্লাস ১৭-এ স্লাইস ১-এ ফিরে আসবে।

ধাপ ২

১১টি অ্যাপের আসল আকার

লাইন গোনা হয়েছে টেস্ট ও মাইগ্রেশন বাদ দিয়ে — `apps/*/`



billing সবচেয়ে বড় — অথচ এটা ব্যবসার অংশ, স্কুলের কাজ নয়। কারণ: সাবস্ক্রিপশন, ট্রায়াল, ইনভয়েস, হোয়াইট-লেবেল, প্ল্যাটফর্ম কনসোল — সব একসাথে।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "তোমার সবচেয়ে বড় ফাইল কোনটা?" যদি উত্তর "views.py, ৮০০ লাইন" হয়, তাহলে ওখানেই ভাঙার দরকার। আগে আকার দেখুন, তারপর দায়িত্ব।

ধাপ ২ · গঠন

apps/ ফোল্ডারের আসল গাছ — পাশে টেস্টের সংখ্যা

১১টি অ্যাপ, ১১টি urls.py, ১০৩টি টেস্ট — গোনা সংখ্যা

```
apps/
  accounts/      362     9 tests
  workspaces/    395    10 tests
  core/          658    19 tests
  academics/     589     4 tests
  students/      355     5 tests
  attendance/    343     5 tests
  lessons/       527     4 tests
  assessments/   569    11 tests
  reports/       219     — tests
  audit/         133     — tests
  billing/     1,330    36 tests

config/
  settings/  base · dev · sqlite · prod
  urls.py    ১১টি include
```

এই গাছ থেকে তিনটি কথা

১. প্রতিটি অ্যাপে `urls.py` আছে — অর্থাৎ ঠিকানা অ্যাপের নিজের, `config`-এর নয়।

২. টেস্ট **অ্যাপের ভেতরেই** থাকে — কোড আর প্রমাণ এক জায়গায়।

৩. `settings/` চারটি ফাইলে ভাগ — পরিবেশ বদলালে শুধু ফাইল বদলায়।

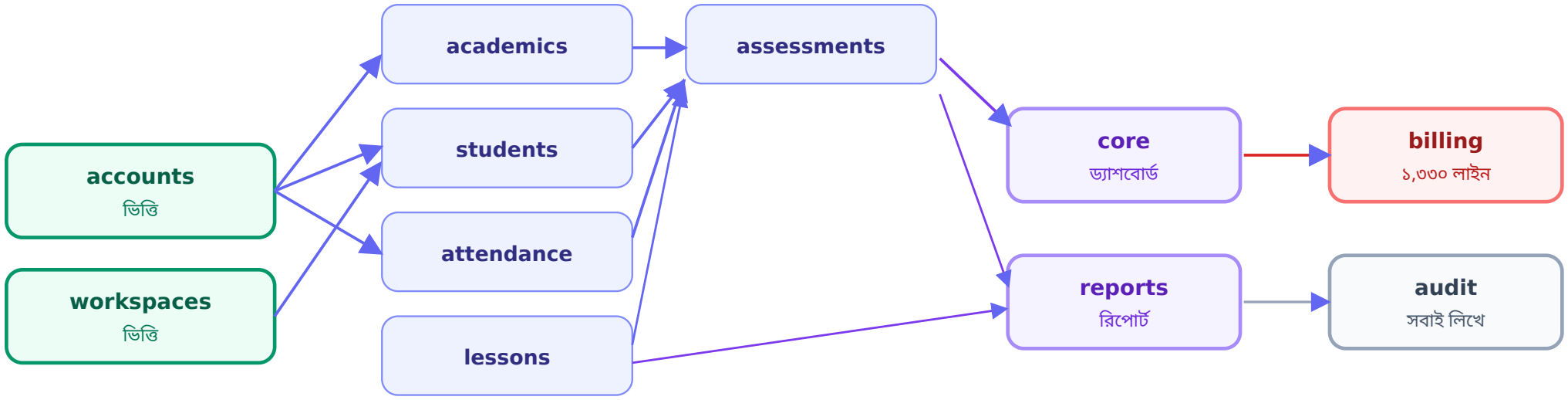
config আর apps আলাদা কেন? `config` হলো বিল্ডিংয়ের বিদ্যুৎ-সংযোগ, `apps` হলো ভেতরের ঘরগুলো। এক জায়গায় গুলিয়ে ফেললে অ্যাপ পুনরায় ব্যবহার করা যায় না।

শিক্ষকের নোট: ক্লাসের আগে টার্মিনালে `find apps -maxdepth 1 -type d` আর `grep -c "def test_" apps/*/tests.py` চালিয়ে আসুন — সংখ্যাগুলো নিজের চোখে দেখলে ছাত্ররা বেশি বিশ্বাস করে।

ধাপ ৩

আসল নির্ভরতার গ্রাফ

তির মানে "কে কাকে import করে" — কোড থেকে যাচাই করা



তিরের দিকটাই আসল: **accounts/workspaces কাউকে import করে না** (প্রায়), আর সবাই ওদের import করে। এটাই ভিত্তি হওয়ার প্রমাণ।

শিক্ষকের নোট: এই গ্রাফটা ড্রয়ারে আঁকার নয়, টার্মিনাল থেকে বের করার — `grep -rn "from apps\." apps/`। ছাত্রদের দেখান কীভাবে কমান্ড দিয়ে নিজের গ্রাফ নিজে বের করবে।

ধাপ ৪

স্তর ভাগ — পাঁচটি স্তর, এক দিকের তির

নিয়ম একটাই: উপরের স্তর নিচের স্তরকে ডাকতে পারে, উল্টোটা নয়

স্তর	অ্যাপ	দায়িত্ব	উপরের স্তরকে কী দেয়
১ · ভিত্তি	accounts · workspaces	কে আপনি, কোন প্রতিষ্ঠানে	ইউজার, সদস্যতা, ভূমিকা
২ · ডোমেইন	academics · students · attendance · lessons · assessments	স্কুলের আসল কাজ	ক্লাস, শিক্ষার্থী, হাজিরা, নম্বর
৩ · সংযোজন	core · reports	একসাথে জোড়া লগিয়ে দেখানো	ড্যাশবোর্ড, সামারি
৪ · ব্যবসা	billing	প্ল্যান, সিট, ইনভয়েস	সীমা ও সাবস্ক্রিপশন
৫ · নিরাপত্তা	audit	কে কী করল, কবে	লগ এন্ট্রি (সবাই লেখে)

বাস্তব সীমা: audit সবাই ব্যবহার করে, কিন্তু কেউ audit-কে ব্যবহার করে না — তাই সে সবার নিচে নেই, **পাশে** আছে। স্তর সবসময় সিঁড়ি নয়, কখনো পাশের দরজা।

শিক্ষকের নোট: ছাত্রদের নিজের প্রজেক্টে এই পাঁচটি স্তর লিখতে বলুন। যার প্রজেক্টে সব কিছু এক স্তরে, তার আর্কিটেকচার এখনো তৈরি হয়নি — শুধু ফোল্ডার আছে।

ধাপ ৫

চক্র (cycle) — আর প্রজেক্টে সেটা কীভাবে ভাঙা হয়েছে

students ↔ billing: দুই দিকের import, তবু সিস্টেম চলে — কীভাবে?

সমস্যাটা কী

students/views.py চায় `seat_block_reason` — সিট ভরা কি না জানতে।
billing/models.py চায় `StudentProfile` — কতজন শিক্ষার্থী আছে গুনতে।

ফাইলের উপরে দুটো import লিখলে Python ঘুরে ঘুরে আটকে যায় — `ImportError`।

সমাধান — ফাংশনের ভেতরে import

```
def add_student(request, slug):
    from apps.billing.services \
        import seat_block_reason
    if seat_block_reason(...):
        return render(request, "seat_full.html")
```

import টা **কল করার সময়** হয়, লোডের সময় নয় — তাই চক্র ভাঙে।

কিন্তু সতর্কতা: **এটা ওষুধ, খাদ্য নয়।** বারবার এমন import দেখলে বুঝবেন দায়িত্ব ভাগ ভুল হয়েছে — সমাধান অ্যাপ সরানো, import লুকানো নয়।

শিক্ষকের নোট: `core/views.py`-তেও একই কৌশল — ফাংশনের ভেতরে মডেল import। ছাত্রদের বলুন: "প্রথমে ভাবো দায়িত্ব কোথায় থাকা উচিত, তারপর দরকার হলে import লুকাও।"

ধাপ ৬

ফাংশনের ভেতরে import — তিনটে বৈধ কারণ

তিনটির বাইরে হলে সেটা লুকোচুরি, ডিজাইন নয়

🔁

১. সত্যিকারের চক্র

দুই অ্যাপ একে অন্যকে দরকার, কিন্তু একটাকে উপরে তুলতেই হবে। তখন **এক দিক** ফাংশনের ভেতরে।

students → billing

🐢

২. ধীর স্টার্টআপ

ভারী লাইব্রেরি (যেমন reportlab) শুধু PDF চাওয়ার সময় দরকার। সার্ভার চালু হওয়ার সময় নয়।

মার্কশিট PDF

🧪

৩. টেস্টে বদলানো

টেস্টের ভেতরে import করলে `mock` বসানো সহজ — কারণ ওই সময়েই নামটা খোঁজা হয়।

১০৩টি টেস্ট

অবৈধ কারণ: "উপরে লিখলে এরর আসছিল" — এররটা এল কেন, সেটাই আসল প্রশ্ন। লুকালে ডিজাইনের সমস্যা থেকে যায়, শুধু দেখতে পান না।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে ফাংশনের ভেতরের import গুনতে — এক-দুটো স্বাভাবিক, দশটা হলে অ্যাপ ভাগ ভুল হয়েছে।

ধাপ ৭

core অ্যাপ — সবকিছুর জোড়া লাগানোর জায়গা

তিনটি ফাইল, তিনটি দায়িত্ব — আর এখানেই সবচেয়ে বড় ফাঁদ

ফাইল	দায়িত্ব	কেন এখানে
`core/navigation.py`	মেনুর একমাত্র তালিকা — কোন ভূমিকা কী দেখবে	একই তালিকা মেনু আর সার্ভারের যাচাইয়ে ব্যবহার হয়, তাই দুই জায়গায় অমিল হওয়া অসম্ভব
`core/permissions.py`	কে কোন প্রতিষ্ঠানে ঢুকতে পারবে	প্রতিটি view-তে আলাদা যাচাই লিখলে একটায় ভুল হলে ফুটো
`core/views.py`	চারটি ড্যাশবোর্ড (F-13 · F-16 · F-29 · F-56)	ড্যাশবোর্ড কোনো এক অ্যাপের নয় — সব অ্যাপের ডেটার সারাংশ

core-এ থাকা উচিত: ক্রস-কাটিং জিনিস — মেনু, পারমিশন, হোম, শেয়ার্ড ট্যাগ।

core-এ থাকা উচিত নয়: উপস্থিতির হিসাব বা নম্বরের নিয়ম — সেগুলো ওই অ্যাপের নিজের কাজ।

শিক্ষকের নোট: `navigation.py`-র কথা মনে করিয়ে দিন — ক্লাস ১-এর "মেনুতে নেই, তবু ঢোকা যায়?" স্লাইডে এটাই প্রমাণ ছিল। এখন তারা দেখছে সেটা কোন অ্যাপে থাকে।

ধাপ ৮

১১টি urls.py — ঠিকানার নিয়ম

সব অ্যাপ একই ছাদে যুক্ত, কিন্তু প্রত্যেকের নিজের পথ

```
# config/urls.py — শুধু জোড়া লাগানো
path("", include("apps.accounts.urls")),
path("", include("apps.workspaces.urls")),
path("", include("apps.core.urls")),
path("", include("apps.academics.urls")),
path("", include("apps.students.urls")),
path("", include("apps.attendance.urls")),
path("", include("apps.lessons.urls")),
path("", include("apps.assessments.urls")),
path("", include("apps.reports.urls")),
path("", include("apps.audit.urls")),
path("", include("apps.billing.urls")),
```

পথের তিনটি পরিবার

পাবলিক — `accounts/login/` · `pricing/`

প্রতিষ্ঠানভিত্তিক — `w/<slug>/students/`

প্ল্যাটফর্ম — `ops/workspaces/` · `workspaces/`

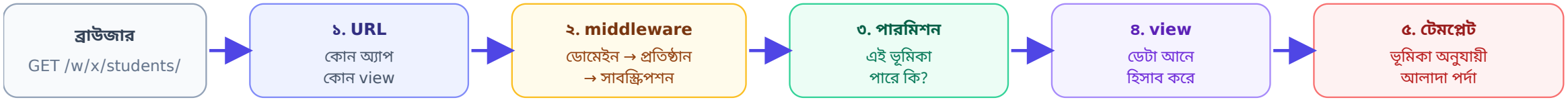
`w/<slug>/` উপসর্গটাই **টেন্যান্টের চিহ্ন** — এই উপসর্গ দেখলেই বোঝা যায় ওই পাতায় প্রতিষ্ঠান যাচাই লাগবে।

শিক্ষকের নোট: জিজ্ঞেস করুন — "কেন সব include-এ prefix খালি?" উত্তর: কারণ প্রতিটি অ্যাপ নিজের পথ নিজে লিখছে; prefix চাপালে URL দুইবার সাজাতে হতো। এটা সচেতন সিদ্ধান্ত, দুর্ঘটনা নয়।

ধাপ ৯ · যাত্রা

একটি অনুরোধের যাত্রা — ৬টি দরজা

ব্রাউজার থেকে পর্দা পর্যন্ত; প্রতিটি দরজায় কেউ না কেউ থামাতে পারে



দরজা ২-এ পরপর তিনটি middleware: CustomDomain → CurrentWorkspace → SubscriptionGate — ক্রম বদলালে নিরাপত্তার ফাঁক তৈরি হয়।

দরজা ৩ ও ৫ মিলে দ্বৈত যাচাই: মেনুতে না দেখানো, আর সার্ভারেও আটকানো — একটার উপর ভরসা করলে ফুটো।

এই ছয়টি দরজা **পর্ব ৪-এর স্লাইস ১**-এ বিস্তারিত আসবে। এখন শুধু মনে রাখুন: **প্রতিটি দরজা একটা অ্যাপের কাজ**।

শিক্ষকের নোট: বোর্ডে এই ছয়টি বাক্স আঁকুন, আর ছাত্রদের জিজ্ঞেস করুন — "ভুল প্রতিষ্ঠানের ছাত্র দেখানো ঠেকানো হয় কোন দরজায়?" উত্তর: দরজা ২ আর ৩, দুই জায়গাতেই।

ধাপ ১০

স্তর ভাগ ঠিক আছে কি না — তিনটি আসল পরীক্ষা

আর্কিটেকচার সুন্দর দেখায় কি না নয়, প্রশ্নটা হলো: টিকে আছে কি না

পরীক্ষা	কীভাবে করবেন	IMS-এ ফল
১. মুছে ফেলার পরীক্ষা	একটা অ্যাপ বাদ দিয়ে সিস্টেম চলে কি?	audit বাদ দিলে চলে (শুধু লগ থাকে না) · accounts বাদ দিলে কিছুই চলে না ✔
২. তিরের দিক পরীক্ষা	নিচের স্তর কি উপরের স্তরকে import করে?	accounts → workspaces (উপরের দিকে এক ধাপ) — একমাত্র ছাড়, কারণ সদস্যতা ছাড়া ইউজার অর্থহীন
৩. নাম বদলের পরীক্ষা	একটা অ্যাপের নাম বদলালে কত জায়গা বদলাতে হয়?	rename TMS → IMS-এ দেখা গেছে: নাম ছড়িয়ে থাকলে ৩টি কমিট লাগে — তাই `PRODUCT_NAME` settings-এ

> **২ নম্বর পরীক্ষায় ছাড় থাকা স্বাভাবিক** — আর্কিটেকচারের নিয়ম ভাঙা যায়, কিন্তু **লিখে রাখতে হয় কেন**। নাহলে পরে কেউ সেটাকে অনুমতি ভেবে আরও ভাঙবে।

শিক্ষকের নোট: "মুছে ফেলার পরীক্ষা" ক্লাসে করে দেখান — একটা অ্যাপের ফোল্ডার সাময়িকভাবে সরিয়ে (git stash দিয়ে) দেখা যায় কী ভাঙে। এটা সবচেয়ে দ্রুত শেখার উপায়।

ধাপ ১১

কখন একটি অ্যাপ ভাঙতে হয় — পাঁচটি সংকেত

অ্যাপ বড় হওয়া দোষ নয়; দায়িত্ব মিশে যাওয়া দোষ

১. দুই কাজ, এক ফাইল

views.py-তে ২০০ লাইনের হিসাব — ওটা `services.py`-র কাজ।

২. দুই দলে ভাগ হয় মেনু

একই অ্যাপের পর্দা যদি দুই সম্পূর্ণ ভিন্ন ভূমিকার হয়, ভাগ করার সময়।

৩. টেস্ট ফাইল ৫০০+ লাইন

টেস্ট ভাগ করা কঠিন মানে কোডও ভাগ করা কঠিন।

৪. একই টেবিল দুই অ্যাপে

দুই জায়গায় একই মডেল — ডেটার মালিক কে, সেটাই অস্পষ্ট।

৫. সাইকেল বাড়ছে

ফাংশনের ভেতরের import প্রতিনিয়ত বাড়ছে — দায়িত্ব ভাগ ভুল।

যখন ভাঙবেন না

শুধু "ফাইল বড়" — এই এক কারণে ভাঙলে ৬টি ফাইলে ৬টি ছোট সমস্যা হবে।

প্রজেক্টে **billing** ১,৩৩০ লাইনে পৌঁছেছে — কারণ কোম্পানির নিজের কাজ আর গ্রাহকের কাজ এক অ্যাপে। এটা আরেকটা ভাঙার প্রার্থী, v2-তে।

শিক্ষকের নোট: শেষ বাক্যটা গুরুত্বপূর্ণ — নিজের প্রজেক্টের সীমাটাও বলুন। ছাত্ররা তখনই বিশ্বাস করে, যখন দেখে আপনি নিজের কাজের দুর্বলতাও বলছেন।

ধাপ ১২

একটা মডেল কোন অ্যাপে থাকবে — সাতটি প্রশ্ন

ডেটাবেস ডিজাইনের আগে দায়িত্ব ঠিক করে নিন

প্রশ্ন	যদি "হ্যাঁ" হয়
এটা কি মানুষ সম্পর্কে (কে আপনি)?	**accounts**
এটা কি প্রতিষ্ঠান/সদস্যতা সম্পর্কে?	**workspaces**
এটা কি কাঠামো (বছর, ক্লাস, বিষয়)?	**academics**
এটা কি একজন শিক্ষার্থীর পরিচয় ও অবস্থা?	**students**
এটা কি দিনের ঘটনা (কে উপস্থিত ছিল)?	**attendance**
এটা কি পড়ানোর বিষয় (সিলেবাস, পাঠ, কাজ)?	**lessons**
এটা কি নম্বর বা ফলাফল?	**assessments**
এটা কি টাকা বা সীমা?	**billing**
হ্যাঁ-র সংখ্যা দুইয়ের বেশি?	**মডেলটা ভাঙতে হবে — দুই অ্যাপের দুই টেবিল**

একই মডেল দুই জায়গায় রাখার প্রলোভন সবচেয়ে বড় ফাঁদ — দুই কপি ডেটা দুই সময়ে বদলায়, তারপর কেউ জানে না কোনটা সত্যি।

শিক্ষকের নোট: শেষ সারিটা সবচেয়ে কাজে লাগে — ছাত্ররা প্রায়ই "Student" মডেলে ক্লাস, ফি, হাজিরা, ফল — সব ঢুকিয়ে ফেলে। এখনই থামান, নইলে ক্লাস ১৪-এ তাদের ERD ভাঙবে।

ধাপ ১৩

আর্কিটেকচার ডায়াগ্রাম — কী দেখাবেন, কী বাদ দেবেন

পর্দায় সব দেখানো যায় না; দেখাতে হয় **সিদ্ধান্ত**টা

দেখাবেন

✓ স্তর ভাগ (ভিত্তি → ডোমেইন → সংযোজন)
✓ এক দিকের তির (কে কাকে ডাকে)
✓ টেন্যান্টের সীমানা — `w/<slug>/`
✓ দুইটি বাইরের জিনিস: ব্রাউজার ও ডেটাবেস
✓ প্রতিটি বাক্সে **লাইনের সংখ্যা** — আকার বোঝায়

বাদ দেবেন

✗ প্রতিটি ফাংশনের নাম
✗ প্রতিটি মডেলের ফিল্ড (ওটা ERD-র কাজ)
✗ লাইব্রেরির ভেতরের গঠন
✗ ফাংশনের ভেতরের import-গুলো
✗ যা কেউ বদলাবে না — যেমন `manage.py`

পরীক্ষা: **এই ডায়াগ্রাম দেখে নতুন কেউ বলতে পারবে — নতুন ফিচারটা কোন বাক্সে যাবে?** না পারলে ডায়াগ্রামটা সাজসজ্জা, নকশা নয়।

শিক্ষকের নোট: ছাত্রদের ডায়াগ্রাম ২ মিনিটে আঁকতে বলুন — সময় কম দিলে তারা আসল সিদ্ধান্তটাই আঁকে, সাজসজ্জা বাদ পড়ে। এটাই সবচেয়ে ভালো পরীক্ষা।

ক্লাসের কাজ

নিজের প্রজেক্টের অ্যাপ ভাগ করুন — ২০ মিনিট

ফলাফল: স্তরের তালিকা + একটি ডায়াগ্রাম

১. দায়িত্বের তালিকা

প্রজেক্টের ১০–১৫টি বড় কাজ লিখুন — ফিচার নয়, **দায়িত্ব**।

২. দলে ভাগ

যেগুলো একসাথে বদলায়, এক অ্যাপে। সাধারণত ৫–৭টি অ্যাপ হয়।

৩. স্তর সাজান

ভিত্তি → ডোমেইন → সংযোজন — এক দিকের তিরে সাজিয়ে নিন।

৪. চক্র খুঁজুন

দুই অ্যাপ একে অন্যকে ডাকছে? দায়িত্ব সরান, import লুকাবেন না।

৫. আঁকুন

প্রতি বাক্সে নাম + কাজের এক লাইন — ২ মিনিটের সীমায়।

যাচাইয়ের প্রশ্ন: আপনার ডায়াগ্রামে **accounts/ভিত্তি** কি কাউকে import করে? যদি করে — তিরটা উল্টো, সেটা ঠিক করুন।

শিক্ষকের নোট: ২০ মিনিট শেষে ২–৩টি ডায়াগ্রাম বোর্ডে টাঙান, একসাথে তিরের দিক মিলিয়ে দেখুন। ভুল তির সবার সামনে ধরুন — এটাই এই কাজটির সবচেয়ে বড় শিক্ষা।

আলোচনা

চারটি প্রশ্ন — আর্কিটেকচার নিয়ে

উত্তর লিখে রাখুন — ক্লাস ১১-এ এগুলোই আবার আসবে

১. আপনার প্রজেক্টে **সবচেয়ে বড় ফাইল** কোনটা, কত লাইন? সে কি এক দায়িত্ব, না তিনটি?

২. আপনার অ্যাপগুলোর মধ্যে **চক্র** আছে? কোন দিকটা ফাংশনের ভেতরে লুকানো?

৩. আপনার **মুছে ফেলার পরীক্ষা** — কোন অ্যাপ বাদ দিলে সিস্টেম এখনো চলবে?

৪. আপনার কোন মডেলটা **দুই অ্যাপের** কাজ করছে? সেটা ভাঙলে কোনটা কোথায় যাবে?

শিক্ষকের নোট: ৪ নম্বরের উত্তর প্রায়ই "Student — ক্লাস, ফি, হাজিরা সব ওখানে" আসবে। বলুন: এটা আজ ঠিক করা সস্তা, ক্লাস ১৪-এর পরে ডেটা ভরা অবস্থায় ঠিক করা অনেক ব্যয়বহুল।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
অ্যাপ = এক দায়িত্ব, ফোল্ডার নয় শুধু	models · views · urls · tests · forms · services — ছয়টি আলাদা কাজ
আকার দেখলে দায়িত্বের ভারসাম্য বোঝা যায়	billing ১,৩৩০ · core ৬৫৮ · academics ৫৮৯ · audit ১৩৩
তির এক দিকে — ভিত্তি কাউকে ডাকে না, সবাই ভিত্তিকে ডাকে	accounts → workspaces (এক ধাপ, লিখিত ছাড়)
চক্র ভাঙা হয় দায়িত্ব সরিয়ে, import লুকিয়ে নয়	students ↔ billing — ফাংশনের ভেতরে import, তিনটি বৈধ কারণ
আর্কিটেকচার যাচাই হয় তিনটি পরীক্ষায়	মুছে ফেলা · তিরের দিক · নাম বদল (TMS→IMS: ৩ কমিট)

শিক্ষকের নোট: তৃতীয় ও চতুর্থ লাইন একসাথে পড়ুন — তির এক দিকের হলে চক্রই জন্মায় না। চক্র দেখা মানে তিরের দিক কোথাও উল্টো হয়েছে — এটাই ক্লাসের মূল কথা।

০৯

পরের ক্লাস — "কে কী পারবে, কে পারবে না"

রোল ম্যাট্রিক্স · ৫×৫ অনুমতি গ্রিড · মেনু ও সার্ভারের দ্বৈত যাচাই

পর্ব ২ · ইঞ্জিনিয়ারিং প্ল্যান · ক্লাস ৯ / ৩৭

কে কী পারবে, কে পারবে না

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.৫ (ভূমিকা ও অনুমতি)
রোল ম্যাট্রিক্স · ৪০৪ বনাম ৪০৩ · এক তালিকা তিন ভোক্তা

২২ মেনু আইটেম · ৫ গ্রুপ · ৪ ভূমিকা

শুরু করুন প্রশ্ন দিয়ে

মেনু থেকে লুকিয়ে দিলেই কি নিরাপত্তা হয়?

প্রথম ক্লাসে এই প্রশ্নটা এসেছিল — আজ উত্তরটা কোডে দেখব

না। মেনু লুকানো, নিরাপত্তা আটকানো।

দুটো আলাদা কাজ — আর দুটোই লাগবে।

কারণ ব্যবহারকারী মেনু দেখে না, **ঠিকানা টাইপ করে**। তাই প্রতিটি অনুরোধে সার্ভারকে নিজে জিজ্ঞেস করতে হয়: "এই মানুষটা এই প্রতিষ্ঠানের সদস্য কি? তার ভূমিকা কি এই কাজ করতে দেয়?"
আজকের ক্লাসে এই দুই প্রশ্নের নকশা — এবং একটি তৃতীয় প্রশ্ন।

শিক্ষকের নোট: কেউ "মেনু লুকালেই তো হয়" বললে বলুন: "ঠিক আছে, তুমি শিক্ষক। এখন ব্রাউজারে `/w/<slug>/audit/` টাইপ করো।" লাইভে দেখালে এই একটা মুহূর্তই পুরো ক্লাসের ভিত্তি হয়ে যাবে।

ধাপ ১

তিনটি প্রশ্ন — প্রতিটি অনুরোধে

`apps/core/permissions.py` -এর নিজের মন্তব্য থেকে

🏢

১. টেন্যান্ট যাচাই

"এই ইউজার কি URL-এ থাকা প্রতিষ্ঠানের **সদস্য**?"

`get_membership(user, workspace)`

না হলে → ৪০৪

🎭

২. ভূমিকা যাচাই

"এই প্রতিষ্ঠানে তার **ভূমিকা** কি এই কাজ করতে দেয়?"

`allowed_roles`

না হলে → ৪০৩

📚

৩. বরাদ্দ যাচাই

"এই শিক্ষক কি **এই ক্লাসে** পড়ান?"

শিক্ষামূলক বস্তুতে তৃতীয় প্রশ্ন

না হলে → ৪০৪ (কিছু নেই)

প্রথম দুই প্রশ্ন **সব** প্রতিষ্ঠান-পাতায়; তৃতীয়টি শুধু যেখানে শিক্ষকের নিজের ক্লাসের কথা — যেমন হাজিরা, নম্বর।

শিক্ষকের নোট: তৃতীয় প্রশ্নটা সবচেয়ে বেশি বাদ পড়ে — ফলে একই প্রতিষ্ঠানের দুই শিক্ষক একে অন্যের ক্লাসের নম্বর বদলাতে পারেন। এটা স্কুলের দৃষ্টিতে গুরুতর ফুটো।

ধাপ ২

NAV ফাইল — ৫ গ্রুপ, ২২ আইটেম, এক তালিকা

`apps/core/navigation.py` — মেনুর একমাত্র সত্য

```
NAV = [
 { "title": None,  "items": [
     _item("Dashboard", …, STAFF+STUDENT) ]},

 { "title": "Teaching", "items": [          # ১০টি
     Students · Classes & subjects
     Attendance · Syllabus · Lesson log
     Homework · Materials · Exams
     Reports · My oversight area ]},

 { "title": "My learning", "items": [        # ৩টি
     My homework · My results · Materials ]},

 { "title": "Administration", "items": [     # ৬টি
     Coordinator scope · Audit log
     Workspace settings · Grading
     Branding · Subscription ]},

 { "title": "Account", "items": [            # ২টি
     My profile · Switch workspace ]},
]
```

প্রতিটি আইটেমে পাঁচটি তথ্য

label — যা দেখা যাবে

icon — কোন চিহ্ন

url_name — কোন রুট (নাম দিয়ে, ঠিকানা দিয়ে নয়)

roles — কারা দেখবে

badge — সংখ্যা দেখানোর জায়গা

url_name কেন? `reverse()` দিয়ে তৈরি হয় — তাই রুট বদলালে মেনু নিজেই বদলে যায়, হাতে ঠিক করতে হয় না।

শিক্ষকের নোট: "Teaching"-এর ১০টি আইটেম বোর্ডে লিখুন — এটাই শিক্ষকের পুরো দিন। তারপর জিজ্ঞেস করুন: এর মধ্যে কোনটা শিক্ষার্থী দেখা উচিত? উত্তর: একটাও না, তার নিজের তিনটি আলাদা গ্রুপে।

ধাপ ৩

ভূমিকা × মেনু গ্রুপ — আসল সংখ্যা

কোড চালিয়ে গোনা: কোন ভূমিকা কতটি আইটেম দেখে

ভূমিকা	Dashboard	Teaching	My learning	Administration	Account	মোট
প্রতিষ্ঠান অ্যাডমিন	১	৯	—	৬	২	**১৮**
কোঅর্ডিনেটর	১	১০	—	—	২	**১৩**
শিক্ষক	১	৯	—	—	২	**১২**
শিক্ষার্থী	১	—	৩	—	২	**৬**
প্ল্যাটফর্ম প্রশাসক	—	—	—	—	—	**০**

কোঅর্ডিনেটর ১৩, শিক্ষক ১২ — পার্থক্য একটাই: "My oversight area"। বাকি সব এক।

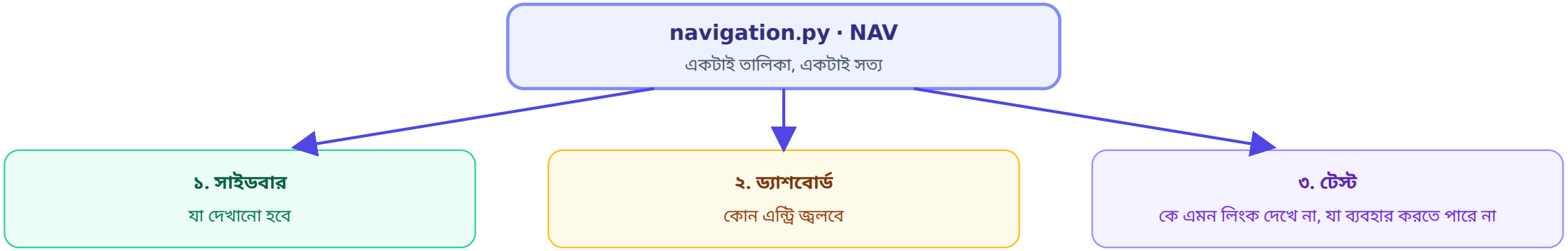
প্ল্যাটফর্ম প্রশাসকের ০ — ইচ্ছাকৃত। তাঁর জগৎ আলাদা: `/ops/`। প্রতিষ্ঠানের মেনুতে তাঁর কিছু নেই, কারণ সেখানে ঢোকার অধিকারও নেই।

শিক্ষকের নোট: "০" ঘরটা নিয়ে আলোচনা করুন — ছাত্ররা ভাবে সুপার-অ্যাডমিন সব পারে। বাস্তবে সে প্রতিষ্ঠানের ভেতরের কোনো ব্যক্তিগত রেকর্ড দেখতে পারে না; পারে শুধু প্রতিষ্ঠান বন্ধ করা বা প্ল্যান বদলানো।

ধাপ ৪

এক তালিকা, তিন ভোক্তা — আর একটি নিরাপদ ব্যর্থতা

ফাইলের মন্তব্যটাই স্লাইডের শিরোনাম: "One definition, three consumers"



নিরাপদ ব্যর্থতা: নতুন পাতা যোগ করে এই ফাইল ভুলে গেলে পাতা মেনুতে আসে না — অর্থাৎ "অদৃশ্য" হয়, "অনিরাপদ" নয়।

কিন্তু সতর্কতা: এই ফাইল শুধু **দেখানো** ঠিক করে। **আটকানো** সবসময় সার্ভারে — দুই জায়গায় একই নিয়ম, কিন্তু দুই দায়িত্ব।

শিক্ষকের নোট: "নিরাপদ ব্যর্থতা" ধারণাটা নতুন — ব্যাখ্যা করুন: ভুল হলে সিস্টেম যেন কম দেখায়, বেশি নয়। বেশি দেখানো মানে ফুটো, কম দেখানো মানে শুধু অসুবিধা।

ধাপ ৫

তিনটি ভূমিকা-দল — কোডের তিনটি ধ্রুবক

প্রতিটি আইটেমে আলাদা করে ভূমিকা লেখা যায়, কিন্তু তিনটি দল বারবার লাগে

⚙️

CONFIG_ROLES

`(ORG_ADMIN,)`

প্রতিষ্ঠান-ব্যাপী কনফিগ: **Branding · Subscription**

কেন শুধু অ্যাডমিন: পরিচয় ও টাকা — দুটোই প্রতিষ্ঠানের সিদ্ধান্ত

🗂️

MANAGER_ROLES

`(ORG_ADMIN,)`

ব্যবস্থাপনার পর্দা: **Coordinator scope · Audit · Settings · Grading**

কোঅর্ডিনেটর এখানে নেই — কারণ নিজের ক্ষমতা নিজে বাড়াতে পারবে না

🎓

STAFF_ROLES

`(ORG_ADMIN, COORDINATOR, TEACHER)`

পড়ানোর কাজ: **Teaching-এর ৯–১০টি আইটেম**

কেন তিনজন: অ্যাডমিনও পড়াতে পারেন, কোঅর্ডিনেটর শিক্ষকও

তিনটি দল = তিনটি **নীতি**। নতুন আইটেম যোগ করার সময় ভাবতে হয় না "কারা দেখবে" — শুধু "কোন দলে"।

শিক্ষকের নোট: জিজ্ঞেস করুন — "কোঅর্ডিনেটরকে MANAGER_ROLES-এ যোগ করলে কী হবে?" উত্তর: সে নিজের স্কোপ বাড়িয়ে ফেলতে পারবে — অর্থাৎ ক্ষমতার সীমা ভেঙে যাবে।

ধাপ ৬

৪০৪ বনাম ৪০৩ — কোনটা কখন

ভুলটা শুধু বার্তা নয়, তথ্য ফাঁসের প্রশ্ন

পরিস্থিতি	কোড	কেন এটাই	ফল
অন্য প্রতিষ্ঠানের সদস্য নন	**৪০৪**	৪০৩ দিলে বোঝা যেত "এই প্রতিষ্ঠান আছে, শুধু আপনি নন" — অর্থাৎ প্রতিষ্ঠানের অস্তিত্ব ফাঁস	"{slug} নেই"
সদস্য, কিন্তু ভূমিকা এই কাজে নয়	**৪০৩**	সে প্রতিষ্ঠানটা জানে, ভেতরে আছে — তাই লুকানোর কিছু নেই, শুধু অনুমতি নেই	"আপনার ভূমিকা এটা করতে দেয় না"
শিক্ষক, কিন্তু ক্লাসটা তার নয়	**৪০৪**	অন্য ক্লাসের অস্তিত্ব জানানোর দরকার নেই	"ক্লাস পাওয়া যায়নি"
লগইনই করা নেই	**৩০২**	লগইন পাতায় পাঠানো — আর ফেরার ঠিকানা সাথে রাখা	`?next=/w/…`

কোডে হুবহু: `raise Http404("Workspace not found.")` — মন্তব্য সহ: *"A platform admin may look at platform-level pages, never at private records of a workspace they do not belong to."*

শিক্ষকের নোট: এই স্লাইডটাই ডেমোতে সবচেয়ে প্রশ্ন তোলে — শিক্ষক লগইন করে অন্য প্রতিষ্ঠানের ঠিকানা টাইপ করলে ৪০৪ পাবেন। লাইভে দেখান।

ধাপ ৭

রিসোর্স ম্যাট্রিক্স — কোন বস্তুতে কে কী

ভূমিকা নয়, **বস্তু** ধরে ভাবুন — এটাই আসল নকশা

বস্তু	প্রশাসক	অ্যাডমিন	কোঅর্ডি.	শিক্ষক	শিক্ষার্থী
প্রতিষ্ঠান	সব দেখে	নিজের	—	—	—
সদস্য	সব	সব	নিজের বিভাগ	—	—
ক্লাস	—	সব	নিজের	নিজের	নিজের
শিক্ষার্থী	—	সব	নিজের বিভাগ	নিজের ক্লাস	শুধু নিজে
উপস্থিতি	—	সব	নিজের + পর্যবেক্ষণ	নিজের ক্লাস	শুধু নিজের
নম্বর	—	গ্রেড স্কেল	—	নিজের ক্লাস	প্রকাশের পরে
বিল	প্ল্যান	সব	—	—	—
অডিট লগ	—	সব	—	—	—

নিয়ম: প্রতিটি ঘরে **"সব"** / **"নিজের"** / **"ধরন"** / **"—"** — চারটি শব্দের বেশি নয়। ঘরে যদি দুই বাক্য লিখতে হয়, নিয়মটা তখনো তৈরি হয়নি।

শিক্ষকের নোট: "নিজের ক্লাস" ঘরটাই যেখানে তৃতীয় প্রশ্ন (assignment) লাগে। ছাত্রদের বলুন এই গ্রিডে নিজের প্রজেক্টের ৫টি বস্তু বসাতে — বেশিরভাগ দেখবে তাদের কোথাও কোথাও "—" লেখার সাহস হচ্ছে না।

ধাপ ৮

ডেকোরেটর — দুই প্রশ্ন এক জায়গায়

প্রতিটি view-তে নিয়ম লিখলে একটায় ভুল হলে ফুটো; তাই নিয়ম এক জায়গায়

```
@workspace_view(Roles.TEACHER,
                Roles.ORG_ADMIN)
def take_attendance(request, slug, …):
    # এখানে পৌঁছানোর মানেই:
    #   ✓ লগইন করা আছে
    #   ✓ প্রতিষ্ঠান আছে ও সক্রিয়
    #   ✓ সদস্য আছেন
    #   ✓ ভূমিকা অনুমতি দেয়
    # …তাই ভেতরে আর যাচাই লিখতে হয় না
    workspace  = request.workspace
    membership = request.membership
```

ডেকোরেটর কী করে (ক্রম অনুযায়ী)

১. লগইন নেই → ৩০২ + `?next=`

২. প্রতিষ্ঠান নেই/বন্ধ → ৪০৪

৩. সদস্য নন → ৪০৪

৪. ভূমিকা মেলে না → ৪০৩

৫. সব ঠিক → `request.workspace` ও `request.membership` বসিয়ে view চালানো

দারুণ কৌশল: ভেতরে `request.workspace` তৈরি হয়ে আসে — ফলে view ভুল করে অন্য প্রতিষ্ঠানের ডেটা আনতে **পারেই না**।

শিক্ষকের নোট: শেষ লাইনটা ভালো করে বলুন — "ভুল করা অসম্ভব বানানো" (make it impossible to get wrong) হলো ভালো আর্কিটেকচারের লক্ষণ। এই ধারণাটা ক্লাস ১৭-এ আবার আসবে।

ধাপ ৯ · প্রমাণ

অনুমতি যাচাই করা হয় টেস্ট দিয়ে — নাম হুবহু

তালিকার তৃতীয় ভোক্তা: "কোন ভূমিকা এমন লিংক দেখে না, যা সে খুলতে পারে না"

টেস্টের নাম (কোডে যেমন আছে)	কী প্রমাণ করে
`test_each_role_is_offered_only_its_own_links`	মেনু ভূমিকা অনুযায়ী সত্যিই ভিন্ন
`test_no_role_sees_a_link_it_cannot_open`	প্রতিটি দেখানো লিংক সত্যিই খোলে
`test_teacher_cannot_open_administration_pages_by_typing_the_url`	ঠিকানা টাইপ করেও লাভ নেই
`test_coordinator_cannot_open_owner_only_pages`	নিজের ক্ষমতা নিজে বাড়ানো যায় না
`test_student_cannot_open_teaching_pages`	শিক্ষার্থীর দরজা বন্ধ
`test_non_member_gets_404_not_403`	প্রতিষ্ঠানের অস্তিত্ব ফাঁস হয় না
`test_cannot_open_another_workspace_by_guessing_its_slug`	slug অনুমান করেও ঢোকা যায় না

লক্ষ্য করুন নামগুলোর ভাষা: **"cannot" আর "gets 404 not 403"** — টেস্টের নামেই নিয়মটা লেখা আছে, তাই ব্যর্থ হলে কী ভাঙল তা সাথে সাথে বোঝা যায়।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে অন্তত এই সাতটির সমতুল্য টেস্ট লিখতে। এগুলো লেখা হয়ে গেলে পারমিশন নিয়ে আর ভয় থাকে না — কারণ প্রতিবার কোড বদলালে টেস্ট বলে দেবে ফুটো হয়েছে কি না।

ধাপ ১০

পাঁচটি সাধারণ ভুল — প্রতিটির পরিণাম

এই পাঁচটি প্রায় সব ছাত্র-প্রজেক্টে পাওয়া যায়

ভুল	কী হয়	ঠিক পথ
শুধু মেনুতে লুকানো	ঠিকানা টাইপ করলেই ঢুকে যায়	view-এ `@workspace_view(...)`
টেমপ্লেটে `if role` দিয়ে লুকানো	HTML-এ শর্ত, সার্ভারে কিছু নেই	শর্ত সার্ভারে, টেমপ্লেটে শুধু সাজসজ্জা
সব জায়গায় ৪০৩	প্রতিষ্ঠানের অস্তিত্ব ফাঁস হয়	অচেনা প্রতিষ্ঠান/ক্লাসে ৪০৪
শিক্ষকের ক্লাস যাচাই না করা	একই প্রতিষ্ঠানের দুই শিক্ষক একে অন্যের নম্বর বদলাতে পারে	তৃতীয় প্রশ্ন: বরাদ্দ যাচাই
`is_superuser` দিয়ে বাইপাস	সুপার-অ্যাডমিন প্রতিষ্ঠানের গোপন রেকর্ড দেখে ফেলে	প্ল্যাটফর্ম প্রশাসকের আলাদা `/ops/` জগৎ

সবগুলোর একটাই কারণ: **"লুকানো" আর "আটকানো" গুলিয়ে ফেলা**। প্রথমটা পর্দার কাজ, দ্বিতীয়টা সার্ভারের।

শিক্ষকের নোট: চতুর্থ ভুলটা ক্লাসে সবচেয়ে বেশি পাওয়া যায় — কারণ ছাত্ররা "একই প্রতিষ্ঠান = সব এক" ভাবে। স্কুলের বাস্তবে শিক্ষক নিজের ক্লাসের বাইরের নম্বর দেখতে পাওয়াটাই বড় সমস্যা।

ক্লাসের কাজ

নিজের অনুমতি গ্রিড বানান — ২০ মিনিট

ফলাফল: একটি টেবিল, যার প্রতিটি ঘরে এক শব্দ

১. বস্তু লিখুন

প্রজেক্টের ৫–৭টি মূল বস্তু: ক্লাস, ছাত্র, অর্ডার, বিল...

২. ভূমিকা লিখুন

সর্বোচ্চ ৫টি — বেশি হলে ভূমিকা ভাগ হয়নি।

৩. ঘর ভরুন

শুধু চার শব্দ: **সব · নিজের · ধরন · —**

৪. দুর্বল ঘর খুঁজুন

যেখানে দুই বাক্য লিখতে হচ্ছে, সেখানে নিয়মটা এখনো স্পষ্ট নয়।

৫. লিখে রাখুন

গ্রিডটা `docs/roles.md` -এ — ক্লাস ১৭-এ এটাই কোড হবে।

যাচাই: আপনার গ্রিডে কি "নিজের" ঘর আছে? না থাকলে তৃতীয় প্রশ্ন (বরাদ্দ যাচাই) নেই — অর্থাৎ আপনার সিস্টেমে সবাই সবাইকে দেখতে পায়।

শিক্ষকের নোট: গ্রিড শেষে ৩ জনকে বোর্ডে ডাকুন। যে গ্রিডে সব ঘরে "সব" লেখা, সেটাই সবচেয়ে বড় শিক্ষার উদাহরণ — ওটা দিয়ে দেখান কী হারাচ্ছে।

আলোচনা

চারটি প্রশ্ন — অনুমতি নিয়ে

উত্তর দিয়ে ক্লাস শেষ করুন

১. আপনার সিস্টেমে **একই প্রতিষ্ঠানের দুই শিক্ষক** একে অন্যের ক্লাসের নম্বর বদলাতে পারবে? কেন?

২. আপনার কোথায় **৪০৪** আর কোথায় **৪০৩** হবে — দুটো উদাহরণ দিন।

৩. আপনার মেনুতে **একই আইটেম** দুই ভূমিকার জন্য ভিন্ন অর্থ বোঝায়? সেটা কি আলাদা করা দরকার?

৪. আপনার সিস্টেমে **সুপার-অ্যাডমিন** আছে? সে কি প্রতিষ্ঠানের ব্যক্তিগত রেকর্ড দেখতে পারে — পারলে কেন?

শিক্ষকের নোট: ৪ নম্বরের উত্তর দিয়ে বোঝা যায় ছাত্র "শক্তি" আর "দায়িত্ব" আলাদা করতে শিখেছে কি না। "আমিই সব, তাই সব পারব" — এটা নকশা নয়, অভ্যাস।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
প্রতিটি অনুরোধে তিনটি প্রশ্ন — টেন্যান্ট · ভূমিকা · বরাদ্দ	`permissions.py`-এর নিজের ডকস্ট্রিং, হুবহু তিনটি প্রশ্ন
মেনু লুকায়, সার্ভার আটকায় — দুটো আলাদা দায়িত্ব	`navigation.py`: "this file only decides what is shown"
এক তালিকা, তিন ভোক্তা + নিরাপদ ব্যর্থতা	সাইডবার · ড্যাশবোর্ড · টেস্ট — ফাইল ভুলে গেলে পাতা অদৃশ্য, অনিরাপদ নয়
অচেনা লোকের জন্য ৪০৪, ভুল ভূমিকার জন্য ৪০৩	`Http404("Workspace not found.")` — অস্তিত্ব ফাঁস না করার সিদ্ধান্ত
ভূমিকা ৪টি, মেনু আইটেম ২২টি, প্রশাসকের ০	অ্যাডমিন ১৮ · কোঅর্ডিনেটর ১৩ · শিক্ষক ১২ · শিক্ষার্থী ৬ · প্রশাসক ০

শিক্ষকের নোট: শেষ লাইনে "প্রশাসকের ০" — এটা নিয়ে শেষ কথা বলুন: "যত কম দেখতে পারে, তত কম ভাঙতে পারে। সর্বশক্তিমান অ্যাডমিন হলো সবচেয়ে বড় ঝুঁকি।"

১০

পরের ক্লাস — "কত দিনে শেষ হবে"

শিডিউল · নির্ভরতার ক্রম · ১২ সপ্তাহের বাজেট ও বাফার

কত দিনে শেষ হবে

সিলেবাস: প্র্যাকটিক্যাল ৩ · টপিক ৩.৬ (শিডিউল ও বাজেট)

নির্ভরতার ক্রম · ১২ সপ্তাহের গ্যান্ট · বাফার

৫ স্লাইস · ১২ সপ্তাহ · ১,৭৫০ লাইন/সপ্তাহ

শুরু করুন প্রশ্ন দিয়ে

"১২ সপ্তাহে হয়ে যাবে" — এই বাক্যের সমস্যা কী?

হাত তুলুন: আপনার প্রজেক্টের শিডিউল কি কাগজে আছে, নাকি মাথায়?

শিডিউল মানে তারিখের তালিকা নয়।

শিডিউল মানে কোন কাজ কোন কাজের পরে।

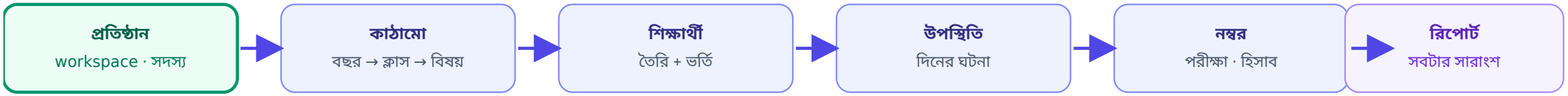
কারণ কাজগুলো **এলোমেলো নয়** — উপস্থিতি নেওয়ার পর্দা বানানো যায় না, যতক্ষণ ক্লাস আর শিক্ষার্থীর টেবিল না দাঁড়ায়। তাই আগে **নির্ভরতার ক্রম**, তারপর তারিখ। উল্টো করলে শিডিউল সুন্দর হয়, কিন্তু কাজ আটকে থাকে।

শিক্ষকের নোট: কেউ "আমার প্রজেক্ট ৪ সপ্তাহে হবে" বললে জিজ্ঞেস করুন: "প্রথম সপ্তাহের শেষে ঠিক কী চলবে?" উত্তর যদি "সব কিছু শুরু করেছি" হয়, তাহলে সপ্তাহ শেষে কিছুই চলবে না।

ধাপ ১

নির্ভরতার ক্রম — কোন কাজ কীসের পরে

তির মানে "আগে এটা, তারপর ওটা" — উল্টো করলে সময় নষ্ট, শুধু নয়, কাজ ভাঙে



উল্টো করার পরিণাম: নম্বরের পর্দা আগে বানালে শিক্ষার্থীর টেবিল পরে বদলাতে হয় → প্রতিটি পর্দা আবার বদলাতে হয়।
উপস্থিতি আগে বানালে ক্লাসের কাঠামো পরে বদলালে সব রেকর্ড নতুন করে মেলাতে হয়।

স্লাইড ১-এর ক্রমটাই আসল: **ভিত্তি → কাঠামো → মানুষ → দৈনন্দিন → ফল → সারাংশ**। তারিখ বসানোর আগে এই ক্রমটা ঠিক করুন।

শিক্ষকের নোট: ব্ল্যাকবোর্ডে এই ছয়টি বাক্স আঁকুন, তারপর ছাত্রদের নিজের প্রজেক্টের কাজগুলোতে বসাতে বলুন। বেশিরভাগ দেখবে তারা "পর্দা বানানো" দিয়ে শুরু করতে চেয়েছিল।

ধাপ ২

১২ সপ্তাহ — আসল গ্যান্ট চার্ট

প্রতিটি সপ্তাহের শেষে সিস্টেম চলে, টেস্ট সবুজ — এটাই শিডিউলের একমাত্র শর্ত



শিক্ষকের নোট: গ্যান্ট চার্টে "স্লাইস" শব্দটা জোর দিন — ফিচারের তালিকা নয়, চলমান সিস্টেমের টুকরো। প্রতিটি স্লাইস শেষে ডেমো দেওয়া যায়, এটাই শিডিউলের আসল নিরাপত্তা।

ধাপ ৩

প্রতিটি স্লাইসে ঠিক কী থাকে

এক স্লাইস = এক সম্পূর্ণ ব্যবহারযোগ্য সিস্টেম, অর্ধেক পর্দা নয়

স্লাইস	কী যোগ হয়	শেষে ব্যবহারকারী কী করতে পারে	প্রমাণ
১	লগইন · প্রতিষ্ঠান · সদস্যতা · ভূমিকা · ৪টি ড্যাশবোর্ড	লগইন করে নিজের প্রতিষ্ঠানে ঢুকে নিজের ভূমিকার ড্যাশবোর্ড দেখতে পারে	১৩ টেস্ট
২	বছর · টার্ম · বিভাগ · ক্লাস · বিষয় · শিক্ষক-বরাদ্দ · শিক্ষার্থী · ভর্তি	প্রতিষ্ঠানের পূর্ণ কাঠামো দাঁড়ায়, শিক্ষার্থী ক্লাসে যুক্ত হয়	৪৮ পর্দা × ৪ ভূমিকা
৩	উপস্থিতি · সিলেবাস · পাঠ-লগ · হোমওয়ার্ক	শিক্ষক প্রতিদিনের কাজ করতে পারে — হাজিরা থেকে হোমওয়ার্ক	শিক্ষকের পূর্ণ দিন
৪	পরীক্ষা · নম্বর · গ্রেড · হিসাব · প্রকাশ · মার্কশিট PDF	নম্বর থেকে ফলাফল পর্যন্ত পুরো পথ — প্রকাশের পরে শিক্ষার্থী দেখে	মার্কশিট PDF
৫	রিপোর্ট · সারাংশ · এক্সপোর্ট · অডিট লগ · পলিসি	প্রতিষ্ঠান চালানোর সিদ্ধান্ত নেওয়ার মতো তথ্য	১০৩ টেস্ট

লক্ষ্য করুন: স্লাইস ১-এ **শিক্ষার্থী নেই**, স্লাইস ২-এ **হাজিরা নেই** — তবু প্রতিটি স্লাইস শেষে সিস্টেম চলে। এটাই "চলমান টুকরো" হওয়ার শর্ত।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "তোমার স্লাইস ১-এ কী থাকবে?" উত্তর যদি "৬টি ফিচার" হয়, জিজ্ঞেস করুন "শেষে ব্যবহারকারী ঠিক কী করতে পারবে?" — উত্তর না এলে স্লাইসটা ভাঙতে হবে।

ধাপ ৪

গতির হিসাব — আসল সংখ্যা দিয়ে

৯,৩২২ লাইন কোড ÷ ১২ সপ্তাহ — কিন্তু এই ভাগটা একা মিথ্যা বলে

📄 **৯,৩২২ লাইন**
Python (মাইগ্রেশন বাদ)

🖼️ **৩,২৬৫ লাইন**
HTML (৭৫ টেমপ্লেট)

🧪 **১০৩ টেস্ট**
১১টি অ্যাপে

🗓️ **১২ সপ্তাহ**
× ৫ দিন × ৪ ঘণ্টা = ২৪০ ঘণ্টা

হিসাব	ফল	কেন বিভ্রান্তিকর
মোট লাইন ÷ মোট ঘণ্টা	**প্রতি ঘণ্টায় ~৫২ লাইন**	শেষ সপ্তাহে ৪০০ লাইন, প্রথম সপ্তাহে ২০ — গড় কিছুই বলে না
স্লাইস ÷ সপ্তাহ	**প্রতি ২.৪ সপ্তাহে ১ স্লাইস**	প্রথম স্লাইস ছোট, দ্বিতীয় স্লাইস সবচেয়ে বড়
টেস্ট ÷ সপ্তাহ	**সপ্তাহে ~৯টি**	বাস্তবে সপ্তাহে ০, তারপর একদিনে ৩০

যেটা কাজে দেয়: "গত সপ্তাহে আমি কতগুলো স্লাইস-শর্ত শেষ করেছি" — শেষ **৩ সপ্তাহের** গড় দিয়ে বাকি সময় হিসাব করুন।

শিক্ষকের নোট: ছাত্রদের বলুন প্রতি সপ্তাহে এক লাইনে লিখতে: "এই সপ্তাহে কী কাজ করল, কী আটকে আছে"। তিন সপ্তাহ পরে ওই তিন লাইনই তাদের সবচেয়ে নির্ভুল শিডিউল।

ধাপ ৫

বাফার — কেন ২৫%, আর কোথায় রাখবেন

বাফার "অলসতার জায়গা" নয় — অজানা জিনিসের দাম

বাফার কোথায় রাখা ভুল

✓ শেষে ৩ সপ্তাহ খালি রাখা — "পরে ঠিক করে নেব"

✗ তখন প্রথম ৯ সপ্তাহেই সব দেরি জমে যায়

✗ আর শেষে বাফারটা কাজে লাগে **দেরি ঢাকতে**, নতুন কিছু শিখতে নয়

সঠিক নিয়ম

প্রতিটি স্লাইসের ভেতরেই বাফার — অর্থাৎ স্লাইস ১-এর কাজ ৪ দিনে ধরুন, ৫ দিনের ঘরে; স্লাইস ২-এর কাজ ৮ দিনে ধরুন, ১০ দিনের ঘরে।

দেরি হলে স্লাইসেই ধরা পড়ে, শেষে নয়।

২৫% কোথা থেকে

IMS-এর আসল টাইমলাইনে **৯,৩২২ লাইন কোড ২ কর্মদিবসে** লেখা হয়েছে — কিন্তু এর মধ্যে **৪টি বাগ সংশোধন** আছে (CSRF, বিলিং, workspace, রেনেম)।

অর্থাৎ "লেখার সময়" আর "ঠিক করার সময়" — মিলিয়ে ২৫–৪০%।

ভয়ংকর হিসাব: ৫৭ ফিচার ÷ ১২ সপ্তাহ = সপ্তাহে ৫টি। কাগজে ঠিক, বাস্তবে অসম্ভব — কারণ ফিচারগুলো সমান নয়।

শিক্ষকের নোট: "বাফার মানে অলসতা নয়" — এই বাক্যটা ক্লাসে দুবার বলুন। ছাত্ররা বাফার রাখে না কারণ মনে করে অধ্যাপক কম পরিশ্রম দেখবেন; আসলে বাফারহীন শিডিউলই সবচেয়ে বেশি মিথ্যা।

ধাপ ৬

সমালোচনামূলক পথ — কোন কাজ দেরি হলে সব দেরি

Critical path: সব কাজ সমান নয়, কয়েকটা পুরো প্রজেক্ট টেনে নেয়



সমান্তরালে চলতে পারে (দেরি হলেও ক্ষতি নেই):

- রিপোর্ট এক্সপোর্ট
- অডিট লগ
- ব্র্যান্ডিং / হোয়াইট-লেবেল
- ব্যাকআপ কনসোল

সমালোচনামূলক পথের মোট = ২২ দিন (কাজের দিন) → প্রায় ৫ সপ্তাহ। বাকি কাজ এই পথের ফাঁকে চলে।

নিয়ম: সমালোচনামূলক পথে একজন মানুষ কাজ করবে — দুই জন দিলে দুই জন একে অন্যের কাজে বাধা দেয়।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন — "তোমার সবচেয়ে বেশি দেরি হওয়া কাজ কোনটা, আর তার পরে কোন কাজ আটকে আছে?" উত্তরই তাদের সমালোচনামূলক পথ।

ধাপ ৭

শিডিউলের চারটি ফাঁদ

প্রতিটি ফাঁদের একটা লক্ষণ আছে — শুরুতেই ধরা যায়

ফাঁদ	লক্ষণ	প্রতিকার
১. ছাত্র-ঘণ্টা ধরে নেওয়া	"দিনে ৮ ঘণ্টা কাজ করব" — ৩ দিনও হয়নি	দিনে **৩–৪ ঘণ্টা** ধরে হিসাব করুন; বাকিটা ক্লাস, পড়া, জীবন
২. সব কাজ সমান ধরা	"৫৭ ফিচার ÷ ১২ সপ্তাহ"	ধরন অনুযায়ী ভাগ: list আধা দিন · form ১ দিন · report ২ দিন · model ৩ দিন
৩. পর্দা দিয়ে শুরু	প্রথম সপ্তাহে সুন্দর ড্যাশবোর্ড, ডেটাবেস পরে	নির্ভরতার ক্রম মানুন: ভিত্তি → কাঠামো → মানুষ → ঘটনা → ফল
৪. শেষ সপ্তাহে ডিপ্লয়	১১তম সপ্তাহেও কোথাও চালু হয়নি	**৮ম সপ্তাহেই** ডিপ্লয় করুন — সস্তায়, কুৎসিতভাবে, কিন্তু চালু

চতুর্থ ফাঁদটা সবচেয়ে মারাত্মক: শেষ সপ্তাহে ডিপ্লয় করতে গেলে প্রথমবারই দেখা যায় `DEBUG=False`-এ সাইট ভাঙে — ঠিক করার সময় তখন আর নেই।

শিক্ষকের নোট: প্রজেক্টে এই ফাঁদ এড়ানো হয়েছে — দিন ১-এই `run.sh` আর `sqlite` settings তৈরি। ছাত্রদের বলুন ৮ম সপ্তাহের ডিপ্লয়টা ক্যালেন্ডারে এখনই লিখে ফেলতে।

ধাপ ৮

বাজেট — টাকা আর সময় একসাথে

ক্লাস ৩-এর ফিজিবিলিটি এখানে শিডিউলে মিশে যায়

খাত	পরিমাণ	নোট
সময় (প্রধান খরচ)	**২৪০ ঘণ্টা**	১২ সপ্তাহ × ৫ দিন × ৪ ঘণ্টা
ডেমো সার্ভার	৳০ – ৳১,৮০০/মাস	ডেমোর জন্য ফ্রি টিয়ারও চলে; প্রোডাকশনে VPS
ডোমেইন	৳১,২০০/বছর	ডেমোতে সাবডোমেইনই যথেষ্ট
ইমেইল	৳০	প্রথমে কনসোল ব্যাকএন্ড (ডেমোতে দেখা যায়)
PDF / Excel	৳০	লাইব্রেরি ওপেন সোর্স
মোট নগদ খরচ (ডেমো)	**৳০ – ৳৩,০০০**	আসল বিনিয়োগ সময়, টাকা নয়

এই কারণেই শিক্ষার্থী-প্রজেক্টে বাজেট আলোচনা হয় **সময়** দিয়ে, টাকা দিয়ে নয়। সময়ই একমাত্র খরচ যা ফেরত পাওয়া যায় না।

শিক্ষকের নোট: কেউ "সার্ভারে টাকা লাগবে" বললে বোঝান: ৳১,৮০০ মাসিক মানে সেমিস্টার জুড়ে ৳৭,২০০ — একটি ল্যাপটপের ১%। তাই অর্থ সমস্যা নয়, সময়ই আসল সীমা।

ধাপ ৯ · প্রমাণ

শিডিউলের রিপোর্টিং — সপ্তাহে এক টেবিল

কেউ জিজ্ঞেস করলে ৩০ সেকেন্ডে উত্তর দেওয়ার মতো ফরম্যাট

সপ্তাহ	লক্ষ্য ছিল	যা শেষ হলো	প্রমাণ
৭	উপস্থিতি + সিলেবাস	উপস্থিতি শেষ, সিলেবাস অর্ধেক	`30_attendance_take.png` · ৫ টেস্ট
৮	সিলেবাস + হোমওয়ার্ক	দুটোই শেষ	`36_syllabus_detail.png` · ৪ টেস্ট
৯	পরীক্ষা সেটআপ	লক্ষ্যের বেশি — নম্বর এন্ট্রিও	`43_marks_entry.png` · ১১ টেস্ট
১০	ফলাফল + PDF	বাকি — PDF রেন্ডারার জটিল	`44_marksheet.png` (আংশিক)

দশম সপ্তাহের সারিটাই সবচেয়ে দরকারি — এখানে সৎভাবে **"বাকি"** লেখা আছে। যে রিপোর্টে কখনো "বাকি" থাকে না, সেটা রিপোর্ট নয়, বিজ্ঞাপন।

শিক্ষকের নোট: ছাত্রদের প্রতিটি সারিতে স্ক্রিনশটের নাম বা টেস্টের নাম লিখতে বাধ্য করুন। প্রমাণ কলাম না থাকলে শিডিউল রিপোর্টিং আত্মপ্রশংসায় পরিণত হয়।

ধাপ ১০

দেরি হলে কী করবেন — পাঁচটি ক্রম

দেরি হবে, সেটা ধরে নিন; প্রশ্ন হলো কোনটা আগে ছাড়বেন

১. আগে বাদ দিন

v1.1-এর কাজ

আমন্ত্রণ · ব্যাকআপ · এক্সপোর্ট

২. সরল করুন

PDF বাদ, প্রিন্ট-ভিউ দিন

এক্সেল বাদ, CSV দিন

৩. কম লোক দেখান

কোঅর্ডিনেটর ভূমিকা

v1.1-এ সরান

৪. কখনোই বাদ দেবেন না

লগইন · টেন্যান্ট · ভূমিকা

অডিট · প্রকাশের নিয়ম

৫. দিন শেষে

ডেমো ডেটা তৈরি করুন

খালি সিস্টেম দেখাবেন না

এই ক্রমটাই সঠিক কারণ ১–৩ বাদ দিলেও সিস্টেম নিরাপদ থাকে, আর ৪ বাদ দিলে সিস্টেম ঝুঁকিপূর্ণ হয়ে যায়।

সবচেয়ে খারাপ সিদ্ধান্ত: সময় বাঁচাতে টেন্যান্ট যাচাই বা পারমিশন বাদ দেওয়া — সেটা সময় বাঁচায় না, বিশ্বাস হারায়।

দ্বিতীয় খারাপ: খালি ডেটাবেস নিয়ে ডেমো দেওয়া — "কিছু নেই" আর "কিছু কাজ করে না" দেখতে একই লাগে।

শিক্ষকের নোট: এই পাঁচটি ক্রম বোর্ডে লিখে রাখুন, সেমিস্টারের শেষে কেউ "আরও দুই সপ্তাহ লাগবে" বললে ওই লেখাটাই আপনাকে সাহায্য করবে — কোনটা ছাড়তে হবে তা নিয়ে বিতর্ক কমবে।

ক্লাসের কাজ

নিজের ১২ সপ্তাহের শিডিউল — ২৫ মিনিট

ফলাফল: একটি টেবিল, যার প্রতিটি সপ্তাহে একটা প্রমাণ

১. স্লাইস ভাগ করুন

৪–৫টি স্লাইস, প্রতিটির শেষে সিস্টেম চলে।

২. ক্রম ঠিক করুন

ভিত্তি → কাঠামো → মানুষ → ঘটনা → ফল।

৩. সপ্তাহ বসান

প্রতি স্লাইসে ২৫% বাফার — বেশি নয়, কমও নয়।

৪. প্রমাণ লিখুন

প্রতি সপ্তাহে একটা টেস্টের নাম বা স্ক্রিনশটের নাম।

৫. ৮ম সপ্তাহে ডিপ্লয়

ক্যালেন্ডারে তারিখ লিখুন — এখনই।

যাচাই: আপনার শিডিউলে কি এমন কোনো সপ্তাহ আছে যার শেষে **কিছুই চলে না**? থাকলে ওই সপ্তাহটাই ভুল করে সাজানো হয়েছে — স্লাইস ভাঙুন।

শিক্ষকের নোট: শেষে ২ জনের শিডিউল বোর্ডে দেখুন — যার প্রতি সপ্তাহে প্রমাণ লেখা আছে, তার শিডিউল বাস্তবে টিকবে; যার শুধু কাজের নাম আছে, সে তৃতীয় সপ্তাহেই পিছিয়ে পড়বে।

আলোচনা

চারটি প্রশ্ন — শিডিউল নিয়ে

পর্ব ২ শেষ হচ্ছে — উত্তরগুলো পর্ব ৩-এ কাজে লাগবে

১. আপনার **স্লাইস ১**-এ কী থাকবে, আর শেষে ব্যবহারকারী ঠিক কী করতে পারবে?

২. আপনার **সমালোচনামূলক পথ** কোন কাজগুলো দিয়ে যায়? দেরি হলে কী আটকাবে?

৩. আপনার শিডিউলে **বাফার** কত? সেটা প্রতিটি স্লাইসের ভেতরে, না শুধু শেষে?

৪. দেরি হলে আপনি **প্রথমে** কোনটা ছাড়বেন? যদি উত্তর "পারমিশন যাচাই" হয় — শিডিউলটা নতুন করে ভাবুন।

শিক্ষকের নোট: ৪ নম্বর উত্তরই পর্ব ২-এর শেষ পরীক্ষা — সেটা ঠিক থাকলে ছাত্র বুঝেছে "কী ছাড়া যায়, কী ছাড়া যায় না"। পরের পর্বে এই বোধটাই ডিজাইনের ভিত্তি হবে।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
শিডিউল আগে নির্ভরতা, পরে তারিখ	ভিত্তি → কাঠামো → মানুষ → ঘটনা → ফল → সারাংশ
স্লাইস মানে চলমান সিস্টেম, কাজের তালিকা নয়	৫ স্লাইস · প্রতিটির শেষে ডেমো দেওয়া যায়
গড় হিসাব মিথ্যা বলে — শেষ ৩ সপ্তাহের গতি ধরুন	৯,৩২২ লাইন ÷ ২৪০ ঘণ্টা = ৫২ লাইন/ঘণ্টা — বাস্তবে অসম
বাফার প্রতিটি স্লাইসের ভেতরে, শেষে নয়	২ দিনের কাজ ৫ দিনের ঘরে — দেরি স্লাইসেই ধরা পড়ে
দেরি হলে ছাড়ার ক্রম আগেই ঠিক করা	v1.1 → সরলীকরণ → কম লোক · কখনোই নয়: টেন্যান্ট/ভূমিকা

শিক্ষকের নোট: এই পাঁচটি লাইনে পর্ব ২ শেষ হচ্ছে। বলুন: "পর্ব ৩-এ আমরা এই শিডিউল ধরে ডিজাইন শুরু করব — ERD, wireframe, আর সিস্টেম ডিজাইন।"

পর্ব ৩

ক্লাস ১১ — "ডেটা কীভাবে ভাবে"

ডিজাইন পর্ব: সিস্টেম ডিজাইন · রিকোয়েস্ট ফ্লো · wireframe · DFD · ERD